Understanding ECRformer-Light

A Beginner-to-Research-Level Guide to Multimodal Satellite Image Cloud Removal

From Neural Network Fundamentals to SAR-Guided Optical Reconstruction, Transformer Attention, Cloud Masks, Loss Functions, and Model Training

Research & Engineering Documentation Team

October 2026

Preface

This textbook provides a comprehensive, self-contained educational curriculum designed to teach the fundamentals and advanced engineering of multimodal satellite image cloud removal from the ground up. Whether you are encountering deep learning and tensors for the very first time or preparing to defend research-level architectural innovations in generative computer vision, this text explains every concept with intuitive everyday analogies, step-by-step numerical worked examples, formal mathematical derivations, and verified line-by-line source-code traceability.

Contents

1 Understanding the Satellite Cloud Removal Problem **1**
- 1.1 Introduction to Earth Observation and Remote Sensing . . . 1
- 1.2 The Two Sister Constellations: Sentinel-1 and Sentinel-2 . . . 1
  - 1.2.1 Sentinel-2: High-Resolution Optical Multispectral Imagery . . . 2
  - 1.2.2 Sentinel-1: Synthetic Aperture Radar (SAR) . . . 2
- 1.3 The Multimodal Reconstruction Challenge . . . 2
- 1.4 Summary of Chapter 1 . . . 3
- 1.5 Practice Exercises . . . 3

2 Understanding Satellite Image Bands and Tensors **5**
- 2.1 Multispectral Bands: Beyond Human Vision . . . 5
  - 2.1.1 Why the Near-Infrared (NIR) Band is Essential . . . 5
- 2.2 From Photons to Digital Tensors . . . 6
- 2.3 The Multimodal Input Tensor: $[B, C, H, W]$ . . . 6
- 2.4 Summary of Chapter 2 . . . 7
- 2.5 Practice Exercises . . . 8

3 Python, PyTorch, and Deep Learning Foundations **9**
- 3.1 Object-Oriented Deep Learning in PyTorch . . . 9
- 3.2 The Anatomy of an `nn.Module` . . . 9
  - 3.2.1 Why Never Call `.forward()` Directly . . . 10
- 3.3 Essential Tensor Operations in Satellite Computer Vision . . . 10
  - 3.3.1 Broadcasting . . . 10
  - 3.3.2 Channel Concatenation (`torch.cat`) . . . 10
- 3.4 The Core PyTorch Training Step . . . 10
- 3.5 Summary of Chapter 3 . . . 11
- 3.6 Practice Exercises . . . 11

4 Neural Networks From Scratch **12**
- 4.1 The Biological and Mathematical Artificial Neuron . . . 12
- 4.2 Essential Non-Linear Activation Functions . . . 12
- 4.3 Forward and Backward Propagation: A Complete Numerical Example . . . 12
- 4.4 From a Single Neuron to ECRformer . . . 14
- 4.5 Summary of Chapter 4 . . . 14
- 4.6 Practice Exercises . . . 15

5 Convolutional Neural Networks and Image Features **16**
- 5.1 Why Fully Connected Networks Fail on Satellite Imagery . . . 16
- 5.2 The Convolution Operation . . . 16
- 5.3 Padding and the Importance of Reflection Padding . . . 17
- 5.4 Modern Upsampling: PixelShuffle vs. Transposed Convolution . . . 18

5.5 Summary of Chapter 5 . . . 18
5.6 Practice Exercises . . . 19

6 The Complete Data Pipeline 20
6.1 End-to-End Architectural Overview . . . 20
6.2 Step-by-Step Trajectory of a Satellite Patch . . . 20
6.3 Failure Modes in Multimodal Data Pipelines . . . 20
6.4 Summary of Chapter 6 . . . 22
6.5 Practice Exercises . . . 22

7 SEN12MS-CR Dataset and Preprocessing 23
7.1 The SEN12MS-CR Benchmark Dataset . . . 23
7.1.1 Structure of a Multimodal Triplet . . . 23
7.2 Dataset Splitting: Eliminating Spatial Leakage . . . 24
7.3 Exact Radiometric Normalization Formulas . . . 24
7.3.1 Optical Reflectance Normalization . . . 24
7.3.2 Sentinel-1 SAR Normalization: `paper_exact` . . . 24
7.4 Summary of Chapter 7 . . . 25
7.5 Practice Exercises . . . 26

8 Cloud Masks and Mask-Guided Learning 27
8.1 What is a Cloud Mask? . . . 27
8.2 The Three-Step Mask Refinement Pipeline . . . 27
8.3 The Three Critical Mask Roles: Never Confuse Them! . . . 28
8.4 Summary of Chapter 8 . . . 29
8.5 Practice Exercises . . . 29

9 Encoder-Decoder Architecture 30
9.1 The Philosophy of Encoder-Decoder Networks . . . 30
9.2 The Complete ECRformer-Light U-Net Hierarchy . . . 30
9.2.1 Stage-by-Stage Spatial and Channel Evolution . . . 30
9.3 Summary of Chapter 9 . . . 33
9.4 Practice Exercises . . . 33

10 Transformers and Attention Mechanisms From Scratch 34
10.1 The Fundamental Limitation of Convolutional Networks . . . 34
10.2 The Attention Mechanism: Queries, Keys, and Values . . . 34
10.3 The Scaled Dot-Product Attention Equation . . . 35
10.4 The Complexity Problem: Spatial Attention vs. Channel Attention . . . 36
10.4.1 The Restormer Breakthrough: Transposed Attention (MDTA) . . . 36
10.5 Summary of Chapter 10 . . . 36
10.6 Practice Exercises . . . 36

11 ECRformer Architecture in Detail 38
11.1 The Core Innovation: Hybrid Multimodal Attention . . . 38
11.2 Module A: Multi-DConv Head Transposed Attention (MDTA) . . . 38
11.3 Module B: Multi-Dilated Window Attention (MDWA) . . . 40
11.4 Module C: Gated Depthwise Feed-Forward Network (GDFN) . . . 40
11.5 Module D: Scale-Decoupled Feature Learning (SDFL) . . . 41
11.6 Module E: Mean-Prior Initialization for the Output Head . . . 41
11.7 Summary of Chapter 11 . . . 41
11.8 Practice Exercises . . . 42

12 Understanding the Loss Functions **43**
12.1 The Loss Function: The Objective of Learning . . . 43
12.2 Why Standard L_1 and L_2 Losses Cause Severe Over-Smoothing . . . 43
12.3 The Production Composite Loss Recipe in `train_ecrformer_v4_gan.py` . . . 44
12.3.1 Pillar 1: Scale-Decoupled L_1 Reconstruction Loss ($\mathcal{L}_{\text{rec}}$) . . . 44
12.3.2 Pillar 2: Cloud Structure and Perceptual Loss ($\mathcal{L}_{\text{structure}}$) . . . 44
12.3.3 Pillars 3 & 4: Adversarial and Feature Matching Losses . . . 44
12.4 Summary of Chapter 12 . . . 45
12.5 Practice Exercises . . . 45

13 Generative Adversarial Networks — GANs From Scratch **46**
13.1 The Intuition of Adversarial Competition . . . 46
13.2 Conditional PatchGAN Architecture . . . 46
13.3 The Verified 11-Channel High-Pass Conditioned Discriminator . . . 46
13.3.1 Exact Channel Breakdown: 11 Input Channels . . . 47
13.3.2 Spectral Normalization . . . 48
13.3.3 Mask-Weighted Hinge Adversarial Loss . . . 48
13.4 Summary of Chapter 13 . . . 48
13.5 Practice Exercises . . . 49

14 Backpropagation Through the Actual Architecture **50**
14.1 The Computational Graph: Forward Pass and Autograd Engine . . . 50
14.2 Gradient Flow Through the Soft-Mask Blend Gate . . . 50
14.3 Gradient Propagation Across Skip Connections and Auxiliary Heads . . . 51
14.3.1 1. Additive Gradient Injection from Auxiliary Heads . . . 51
14.3.2 2. Bifurcation at Skip Connections . . . 51
14.4 The 2× Differential Learning Rate on SAR Stem . . . 52
14.5 Summary of Chapter 14 . . . 52
14.6 Practice Exercises . . . 52

15 Training Loop and Experiment Configuration **54**
15.1 The Blueprint of a Production Training Pipeline . . . 54
15.2 DataLoader Engineering: Maximizing GPU Throughput . . . 54
15.3 Mixed Precision Training with Automatic Mixed Precision (AMP) . . . 55
15.4 The Role of Gradient Norm Clipping . . . 55
15.5 Exponential Moving Average (EMA) of Weights . . . 55
15.6 Summary of Chapter 15 . . . 56
15.7 Practice Exercises . . . 56

16 Evaluation Metrics and Result Interpretation **57**
16.1 The Multidimensional Challenge of Visual Evaluation . . . 57
16.2 Core Mathematical Metrics . . . 57
16.2.1 Peak Signal-to-Noise Ratio (PSNR) . . . 57
16.2.2 Structural Similarity Index Measure (SSIM) . . . 57
16.2.3 Detail Ratio: High-Frequency Energy Retention . . . 58
16.3 The Illusion of Global Metrics: Why Cloud Stratification is Mandatory . . . 58
16.4 The Perception-Distortion Trade-Off . . . 58
16.5 Summary of Chapter 16 . . . 59
16.6 Practice Exercises . . . 59

17 Inference and Final Satellite Image Reconstruction **60**
17.1 Transitioning from Training to Production Deployment . . . 60
17.2 The Output Reconstruction Equation . . . 60
17.2.1 Why Pure Neural Outputs are Undesirable on Clear Terrain . . . 60
17.3 Sliding Window Inference for Large-Scale Satellite Scenes . . . 60
17.4 Radiometric Post-Processing and False-Color Visualization . . . 61
17.5 Summary of Chapter 17 . . . 61
17.6 Practice Exercises . . . 62

18 Understanding the Entire Codebase **63**
18.1 Repository Architecture Overview . . . 63
18.2 Master Component Traceability Table . . . 63
18.3 Developer's Diagnostic Guide: Where to Debug Specific Failures . . . 63
18.4 Summary of Chapter 18 . . . 65
18.5 Practice Exercises . . . 65

19 Common Problems, Debugging, and Failure Analysis **66**
19.1 The Reality of Satellite Deep Learning Debugging . . . 66
19.2 The 16 Critical Failure Modes and Proven Remedies . . . 66
19.2.1 1. Optical Radiometric Normalization Mismatch . . . 66
19.2.2 2. Swapped Channel Order . . . 66
19.2.3 3. Memory Layout / Dimension Transposition . . . 66
19.2.4 4. Inverted Cloud Mask . . . 67
19.2.5 5. Misaligned SAR and Optical Imagery . . . 67
19.2.6 6. Noisy or Seasonally Shifted Reference Targets . . . 67
19.2.7 7. Over-Smoothed Outputs (Low Detail Ratio) . . . 67
19.2.8 8. Global Color / Radiometric Drift . . . 67
19.2.9 9. Unstable GAN Training and Mode Collapse . . . 68
19.2.10 10. Loss Plateaus Early in Training . . . 68
19.2.11 11. Exploding or Vanishing Gradients . . . 68
19.2.12 12. Overfitting (Train Loss Falls, Val Loss Rises) . . . 68
19.2.13 13. Spatial Data Leakage . . . 68
19.2.14 14. Metric Calculation Range Errors . . . 68
19.2.15 15. Visible Perimeter Seams in Blended Outputs . . . 69
19.2.16 16. Out of Memory (OOM) GPU Crashes . . . 69
19.3 Summary of Chapter 19 . . . 69
19.4 Practice Exercises . . . 69

20 Understanding the Novel Modifications **70**
20.1 Beyond the Baseline: Why Architectural Adaptations Were Essential . . . 70
20.2 Innovation 1: The 11-Channel High-Pass Conditioned Discriminator . . . 70
20.2.1 The Problem . . . 70
20.2.2 The Mathematical Solution . . . 70
20.2.3 Impact . . . 70
20.3 Innovation 2: Scale-Decoupled L_1 Supervision . . . 71
20.3.1 The Problem . . . 71
20.3.2 The Mathematical Solution . . . 71
20.4 Innovation 3: Dual MDTA + MDWA Hybrid Attention in Every Block . . . 71
20.4.1 The Problem . . . 71
20.4.2 The Architectural Solution . . . 71
20.5 Innovation 4: Opaque Cloud Core Cutout Augmentation . . . 71
20.5.1 The Problem . . . 71

20.5.2 The Implementation . . . 71
20.6 Innovation 5: 2× Differential Learning Rate on SAR Stem . . . 72
20.6.1 The Optimization Strategy . . . 72
20.7 Innovation 6: Soft-Mask Blending Gate with Mean-Prior Initialization . . . 72
20.7.1 The Blending Solution . . . 72
20.8 Summary of Chapter 20 . . . 72
20.9 Practice Exercises . . . 72

21 Ablation Studies and Scientific Evaluation **74**
21.1 The Scientific Method in Deep Learning Research . . . 74
21.2 Systematic Experimental Evolution Across Four Stages . . . 74
21.3 In-Depth Analysis of Ablation Results . . . 74
21.3.1 Stage 1 vs. Stage 4: Deconstructing the "PSNR Trap" . . . 74
21.3.2 The Impact of Intermediate SDFL Auxiliary Supervision . . . 74
21.3.3 The Impact of Soft-Mask Blending . . . 74
21.4 Computational Efficiency and Parameter Footprint . . . 76
21.5 Summary of Chapter 21 . . . 76
21.6 Practice Exercises . . . 76

22 A Complete Worked Example **77**
22.1 Synthesizing the Pipeline with Concrete Numbers . . . 77
22.2 The Micro-World Setup . . . 77
22.2.1 Input Data Values . . . 77
22.3 The Miniature Multimodal Architecture . . . 78
22.4 Step 1: The Forward Pass . . . 78
22.5 Step 2: Loss Evaluation . . . 78
22.6 Step 3: The Backward Pass (Autograd Derivatives) . . . 79
22.7 Step 4: The Optimizer Parameter Update . . . 79
22.8 Step 5: Verifying the Update on Step 2 . . . 79
22.9 Summary of Chapter 22 . . . 80
22.10Practice Exercises . . . 80

23 Frequently Asked Questions **81**
23.1 Core Conceptual and Practical Questions . . . 81
23.2 Summary of Chapter 23 . . . 83
23.3 Practice Exercises . . . 84

24 Glossary and Learning Roadmap **85**
24.1 Comprehensive Technical Glossary . . . 85
24.2 Five-Stage Learning Roadmap for Aspiring Researchers . . . 87
24.3 Summary of Chapter 24 . . . 87
24.4 Practice Exercises . . . 87

Implementation Details and Traceability Audit **89**
24.5 Component Implementation Status Matrix . . . 89
24.6 Exact Parameter Breakdown Audit . . . 89
24.7 Detailed Parameter Counts of Generator Sub-Modules . . . 89

Chapter 1

Understanding the Satellite Cloud Removal Problem

1.1 Introduction to Earth Observation and Remote Sensing

Remote sensing is the science of acquiring information about the Earth's surface from a distance, typically via sensors mounted on satellites orbiting hundreds of kilometers above our planet. Every second, constellations of Earth observation (EO) satellites orbit the globe, recording reflections of sunlight, thermal emissions, and active microwave pulses. These measurements enable environmental scientists, agronomists, urban planners, and disaster responders to monitor agricultural yields, track deforestation in the Amazon, assess flood damage, and monitor polar ice caps.

However, an omnipresent optical obstacle stands between satellite optical sensors and the Earth's surface: **clouds**. According to global meteorological estimates, over 67% of the Earth's surface is covered by clouds at any given moment. In tropical regions, persistent cloud cover frequently exceeds 80% throughout the year, rendering conventional optical imagery unusable for months at a time.

Concept: Everyday Analogy: The Opaque Shower Curtain

Imagine you are looking into a room through a frosted glass window or an opaque shower curtain. Sunlight can illuminate the shower curtain, but you cannot see the floor tiles or the objects resting behind it.

- **Removing visible haze (thin cloud):** Like wiping light steam off the glass; the underlying objects are still dimly visible through the mist.
- **Reconstructing thick cloud cover:** The objects behind the curtain are completely obscured. No amount of contrast enhancement can reveal them because the optical light from the floor never escaped past the curtain to reach your eye!

1.2 The Two Sister Constellations: Sentinel-1 and Sentinel-2

The European Space Agency (ESA) Copernicus Programme operates two complementary satellite constellations that form the backbone of modern multimodal satellite vision:

1.2.1 Sentinel-2: High-Resolution Optical Multispectral Imagery

Sentinel-2 carries a Multispectral Instrument (MSI) that passively measures sunlight reflected from the Earth across 13 spectral bands, ranging from visible blue (490 nm) to shortwave infrared (2190 nm). Because it relies on solar illumination:

- It captures rich natural colors, vegetation vitality, and surface materials.
- It **cannot** see through clouds, dense smoke, or fog, because atmospheric water droplets scatter and absorb shortwave optical photons.
- It cannot acquire data at night.

1.2.2 Sentinel-1: Synthetic Aperture Radar (SAR)

Sentinel-1 carries a C-band (5.405 GHz, $\lambda \approx 5.6$ cm) Synthetic Aperture Radar (SAR) sensor. Unlike optical sensors, Sentinel-1 is an **active** sensor: it transmits its own microwave electromagnetic pulses toward the ground and measures the amplitude and phase of the backscattered signal that reflects back to the satellite antenna.

- **Cloud Penetration:** The microwave wavelength (≈ 5.6 cm) is thousands of times larger than atmospheric cloud water droplets (≈ 10 μm). Consequently, C-band radar waves travel straight through clouds, rain, and haze without attenuation!
- **Physical Sensing:** SAR does not capture color or optical reflectance. Instead, it measures surface roughness, geometric geometry, moisture content, and dielectric properties.
- **Polarization Channels:** Sentinel-1 typically records in two polarization modes:
  1. **VV (Vertical transmit, Vertical receive):** Highly sensitive to flat surfaces, water roughness, and soil scattering.
  2. **VH (Vertical transmit, Horizontal receive):** Cross-polarization, highly sensitive to volume scattering from forest canopies, crops, and complex urban geometry.

1.3 The Multimodal Reconstruction Challenge

Cloud removal is often framed as an **image-to-image translation** task. However, it is fundamentally different from single-image optical super-resolution or denoising.

When a cloud is completely opaque, the optical information under the cloud is strictly zero. If a deep learning model is given only the cloudy optical image, it must guess what lies beneath based on contextual statistical priors (which leads to severe blurring and hallucinations).

By providing co-registered **Sentinel-1 SAR data** alongside the cloudy optical image, the neural network receives genuine, real-time physical evidence of the ground surface beneath the clouds:

$$\mathbf{Y}_{\text{clear}} = \mathcal{G}_\theta\big(\mathbf{X}_{\text{cloudy}}, \mathbf{X}_{\text{SAR}}, \mathbf{M}_{\text{cloud}}\big) \tag{1.1}$$

where:

- $\mathbf{X}_{\text{cloudy}} \in \mathbb{R}^{4\times H\times W}$ is the input cloudy optical image (containing Red, Green, Blue, NIR).
- $\mathbf{X}_{\text{SAR}} \in \mathbb{R}^{2\times H\times W}$ is the coincident Sentinel-1 radar backscatter (VV and VH polarizations).
- $\mathbf{M}_{\text{cloud}} \in [0,1]^{1\times H\times W}$ is the cloud mask indicating confidence of cloud contamination.
- $\mathcal{G}_\theta$ is our deep neural network (ECRformer-Light) parameterized by weights θ.
- $\mathbf{Y}_{\text{clear}} \in \mathbb{R}^{4\times H\times W}$ is the reconstructed clear-sky surface reflectance.

Mathematical Derivation & Numerical Example: A Simple Numerical Matrix Example

Consider a single 2×2 pixel region covered by a cloud:

$$\mathbf{X}_{\text{cloudy}} = \begin{bmatrix} 0.95 & 0.92 \\ 0.98 & 0.94 \end{bmatrix} \quad \text{(high optical brightness, saturated cloud)}$$

Optical data tells us nothing about the underlying terrain. But the co-registered SAR data provides:

$$\mathbf{X}_{\text{SAR, VV}} = \begin{bmatrix} -6.5 & -6.2 \\ -18.4 & -19.1 \end{bmatrix} \text{ dB}, \quad \mathbf{X}_{\text{SAR, VH}} = \begin{bmatrix} -14.1 & -13.8 \\ -25.2 & -26.0 \end{bmatrix} \text{ dB}$$

The top row has high backscatter (characteristic of an urban built-up area), while the bottom row has very low backscatter (smooth calm water or tarmac). The neural network learns to map these SAR signatures to appropriate urban and water optical reflectances:

$$\hat{\mathbf{Y}} = \begin{bmatrix} 0.12 & 0.14 \\ 0.03 & 0.03 \end{bmatrix}$$

Radar signals break the ambiguity of the opaque cloud!

1.4 Summary of Chapter 1

1. Remote sensing captures the Earth from orbit, but clouds obscure over two-thirds of the planet at any moment.
2. Thin clouds (haze) attenuate optical signals; thick clouds completely destroy optical surface information.
3. Sentinel-2 provides rich multispectral optical imagery but cannot see through clouds.
4. Sentinel-1 emits C-band microwaves that penetrate clouds effortlessly, capturing surface geometry, roughness, and moisture via VV and VH polarizations.
5. Multimodal learning fuses radar and optical data so the model can reconstruct the true ground surface beneath clouds without blind hallucination.

1.5 Practice Exercises

1. Why does Sentinel-1 C-band radar penetrate clouds while Sentinel-2 optical light cannot?
2. If a cloud is 100% opaque, can an optical-only neural network reconstruct the exact pixel colors beneath it? Explain why or why not.
3. What physical properties of the Earth's surface are captured by Sentinel-1 VV and VH polarizations?

Solutions

1. The wavelength of Sentinel-1 C-band microwaves (≈ 5.6 cm) is thousands of times larger than the diameter of atmospheric cloud droplets (≈ 10 μm), allowing waves to pass through without Rayleigh scattering. Optical wavelengths (0.4–0.8 μm) are smaller than or comparable to cloud droplet sizes, causing severe Mie and geometric scattering.

2. No. If a cloud is 100% opaque, the ground-reflected photons never reached the sensor. Any optical-only model is performing ill-posed inpainting based on statistical guessing from surrounding context, which cannot know the true underlying terrain.

3. VV polarization is sensitive to surface roughness, soil moisture, and dihedral corner reflections. VH cross-polarization is sensitive to volume scattering from vegetated canopies, crop structures, and complex geometric surfaces.

Chapter 2

Understanding Satellite Image Bands and Tensors

2.1 Multispectral Bands: Beyond Human Vision

Human vision is trichromatic: our retinas contain photoreceptor cone cells sensitive to Red ($\approx$ 650 nm), Green ($\approx$ 550 nm), and Blue ($\approx$ 450 nm) wavelengths. When viewing a digital photograph on a computer screen, each pixel is composed of these three primary additive channels (RGB).

Earth observation satellites, however, are not constrained by human biology. The Sentinel-2 Multispectral Instrument records radiation across 13 distinct bands, capturing crucial non-visible electromagnetic spectra:

Band	**Band Name**	**Center λ (nm)**	**Resolution**	**Primary Physical Role**
`B02`	Blue	490	10 m	Atmosphere/aerosol scattering, clear water bodies
`B03`	Green	560	10 m	Vegetation peak green reflectance, plant health
`B04`	Red	665	10 m	Chlorophyll absorption in healthy vegetation
`B08`	Near-Infrared (NIR)	842	10 m	Internal leaf cellular structure reflectance, water boundaries

Table 2.1: The four 10-meter spatial resolution optical bands utilized in this project from Sentinel-2 Level-1C/2A products.

2.1.1 Why the Near-Infrared (NIR) Band is Essential

In the visible spectrum, healthy green leaves appear green because chlorophyll pigments absorb Red (`B04`) and Blue (`B02`) light to power photosynthesis, while reflecting a modest fraction of Green (`B03`) light ($\approx$ 10–15%).

In the Near-Infrared spectrum (`B08`), however, spongy mesophyll cells inside healthy leaves reflect up to 50–60% of incoming radiation to prevent thermal overheating. Water bodies, on the other hand, absorb almost 100% of NIR radiation, appearing jet black. Including `B08` allows our neural network to:

1. Compute the Normalized Difference Vegetation Index: $\text{NDVI} = \frac{\text{B08}-\text{B04}}{\text{B08}+\text{B04}}$.
2. Reliably distinguish vegetated crops from bare soil, and water coastlines from land, even when haze distorts visible colors.

2.2 From Photons to Digital Tensors

When satellite sensors measure surface radiance, the values are initially recorded as integer **Digital Numbers** (DN). In Sentinel-2 processing pipelines:

$$\rho = \frac{\text{DN}}{10000.0} \tag{2.1}$$

where $\rho \in [0, 1]$ represents dimensionless **Bottom-of-Atmosphere (BOA) Surface Reflectance**. A value of 0.0 means total absorption, while 1.0 represents total reflection.

2.3 The Multimodal Input Tensor: $[B, C, H, W]$

In PyTorch and modern deep learning frameworks, multidimensional arrays of numbers are called **tensors**. Image batches are organized using the standard **channel-first** layout:

$$\mathbf{X} \in \mathbb{R}^{B \times C \times H \times W} \tag{2.2}$$

Let us break down each dimension for our concrete project tensor: $[2, 7, 128, 128]$.

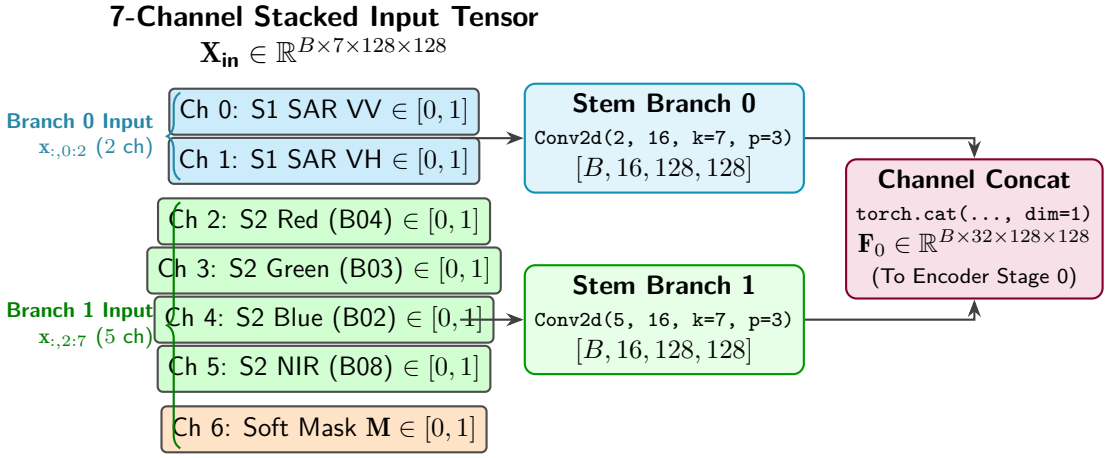


Figure 2.1: Channel layout and modality splitting inside `ECRformerAdaptedModel.forward()` and `DecoupledEncoder.forward()`. The 7-channel input tensor is sliced along dimension $C = 1$ into 2 SAR channels (indices `0:2`) and 5 optical+mask channels (indices `2:7`), projected independently into 16 feature maps each, and concatenated into 32 channels.

- **Batch Size ($B = 2$):** The number of independent geographical image patches processed concurrently in a single forward/backward pass on the GPU. Processing multiple patches simultaneously stabilizes gradient estimates.
- **Channels ($C = 7$):** The number of stacked physical feature maps:
  - `Channel 0`: Sentinel-1 SAR VV backscatter, normalized to $[0, 1]$.
  - `Channel 1`: Sentinel-1 SAR VH backscatter, normalized to $[0, 1]$.
  - `Channel 2`: Sentinel-2 Red band (`B04`), reflectance in $[0, 1]$.
  - `Channel 3`: Sentinel-2 Green band (`B03`), reflectance in $[0, 1]$.
  - `Channel 4`: Sentinel-2 Blue band (`B02`), reflectance in $[0, 1]$.
  - `Channel 5`: Sentinel-2 Near-Infrared band (`B08`), reflectance in $[0, 1]$.

  - `Channel 6`: Cloud Mask $\mathbf{M} \in [0, 1]$, where 1.0 denotes thick cloud and 0.0 denotes clear sky.

- **Height ($H = 128$) and Width ($W = 128$):** The spatial dimensions of each patch. At 10 meters per pixel, a 128×128 patch covers exactly 1.28 km $\times$ 1.28 km = 1.6384 km^2 of the Earth's surface!

Mathematical Derivation & Numerical Example: Memory Footprint Calculation for a Batch

How much VRAM does a single input batch consume? For a single-precision 32-bit floating point tensor (`torch.float32`, 4 bytes per float):

$$\text{Total elements} = B \times C \times H \times W = 2 \times 7 \times 128 \times 128 = 229{,}376 \text{ floats}$$

$$\text{Memory (bytes)} = 229{,}376 \times 4 \text{ bytes} = 917{,}504 \text{ bytes} \approx 0.875 \text{ MB}$$

While the input itself is small (< 1 MB), intermediate activations generated during transformer attention across multiple layers expand the GPU memory requirements during training to several gigabytes.

Code Implementation Traceability: Tensor Slicing in `models/ecrformer_adapted.py`

In our verified implementation, the model receives this 7-channel tensor and splits it internally inside `ECRformerAdaptedModel.forward()`:

```
# S1 SAR channels (Indices 0 and 1)
s1 = x[:, 0:2, :, :]           # Shape: [B, 2, H, W]

# Cloudy S2 optical channels (Indices 2, 3, 4, 5)
s2_cloudy = x[:, 2:6, :, :]    # Shape: [B, 4, H, W]

# Cloud mask channel (Index 6)
mask = x[:, 6:7, :, :]         # Shape: [B, 1, H, W]
```

Notice how clean Python array slicing preserves the separate physical modalities for specialized sub-networks!

2.4 Summary of Chapter 2

1. Sentinel-2 provides multispectral data; our project utilizes Red (`B04`), Green (`B03`), Blue (`B02`), and Near-Infrared (`B08`) at 10 m resolution.
2. Near-Infrared is vital for mapping vegetation vitality and sharp water boundaries that are hidden by clouds.
3. Input data is formatted as a 4-dimensional tensor $[B, C, H, W]$ in channel-first format.
4. The 7 channels consist of 2 SAR channels (VV, VH), 4 optical channels (RGB+NIR), and 1 cloud mask channel.
5. Each 128×128 patch spans 1.64 km^2 of terrain on Earth.

2.5 Practice Exercises

1. Calculate the total number of numbers in a tensor of shape $[4, 7, 256, 256]$.
2. Why does a clear water lake appear dark in the Near-Infrared band (`B08`) but bright in the Blue band (`B02`)?
3. Write the PyTorch slicing syntax to extract only the Near-Infrared channel from an input tensor `x` of shape $[B, 7, H, W]$.

Solutions

1. $4 \times 7 \times 256 \times 256 = 1{,}835{,}008$ numbers (approx 7.0 MB in float32).
2. Liquid water absorbs long-wavelength near-infrared photons almost entirely (reflectance ≈ 0), making lakes appear black in `B08`. Shorter blue wavelengths penetrate water or scatter off surface molecules, yielding measurable reflectance in `B02`.
3. `nir = x[:, 5:6, :, :]` (or `x[:, 5, :, :]` to drop the channel dimension).

Chapter 3

Python, PyTorch, and Deep Learning Foundations

3.1 Object-Oriented Deep Learning in PyTorch

PyTorch is a modern, dynamic scientific computing library that combines multidimensional tensor mathematics on NVIDIA CUDA GPUs with an automatic differentiation engine (`torch.autograd`).

Every neural network component in PyTorch is implemented as an Object-Oriented class that inherits from the fundamental base class: `torch.nn.Module`.

Concept: Analogy: A Factory Assembly Line Worker

Think of an `nn.Module` as a specialized worker on an assembly line:

- **Internal tools (Parameters):** The worker has specific adjustable tools (weights and biases) initialized in the `__init__()` method.
- **Action (`forward()`):** When raw materials (input tensors) arrive on the conveyor belt, the worker applies their tools to transform the material and passes it to the next workstation.
- **Fine-tuning (Gradients):** At the end of the shift, quality control provides feedback on errors, and the worker slightly adjusts their tool settings (updating weights via backpropagation) to perform better tomorrow!

3.2 The Anatomy of an `nn.Module`

Here is how a clean, minimal PyTorch module is constructed:

```
import torch
import torch.nn as nn

class SimpleModalityStem(nn.Module):
    def __init__(self, in_channels=2, out_channels=16):
        super(SimpleModalityStem, self).__init__()
        # Define internal trainable layers
        self.conv = nn.Conv2d(in_channels, out_channels, kernel_size=3,
            padding=1)
        self.relu = nn.ReLU(inplace=True)

    def forward(self, x):
```

```
        # x arrives with shape: [Batch, InChannels, Height, Width]
        feat = self.conv(x)
        out = self.relu(feat)
        return out
```

3.2.1 Why Never Call `.forward()` Directly

In Python, one calls the instantiated model like a function:

```
stem = SimpleModalityStem(2, 16)
output = stem(my_input_tensor) # Correct: invokes __call__()
```

Under the hood, PyTorch's `__call__` method registers forward pre-hooks, execution profiling, and backward autograd hooks before calling `forward(x)`. Calling `stem.forward(x)` directly bypasses these crucial internal mechanisms!

3.3 Essential Tensor Operations in Satellite Computer Vision

3.3.1 Broadcasting

When performing arithmetic operations between tensors of different shapes, PyTorch automatically stretches (broadcasts) the smaller tensor across the missing or singleton dimensions without copying data in memory.

Mathematical Derivation & Numerical Example: Numerical Broadcasting Example

Suppose we have a predicted optical feature map $\mathbf{P} \in \mathbb{R}^{2\times4\times128\times128}$ and a single-channel cloud mask $\mathbf{M} \in \mathbb{R}^{2\times1\times128\times128}$. We evaluate:

$$\mathbf{Y} = \mathbf{M} \odot \mathbf{P}$$

Along dimension $C = 1$, $\mathbf{M}$ has 1 channel while $\mathbf{P}$ has 4 channels. PyTorch broadcasts $\mathbf{M}$ identically across all 4 channels (Red, Green, Blue, NIR):

$$\mathbf{Y}_{b,c,h,w} = \mathbf{M}_{b,0,h,w} \times \mathbf{P}_{b,c,h,w} \quad \text{for each } c \in \{0, 1, 2, 3\}$$

3.3.2 Channel Concatenation (`torch.cat`)

Combining different physical modalities (such as SAR features and optical features) is achieved by stacking tensors along the channel dimension ($C = 1$):

```
# s1_feat: [B, 16, H, W]
# s2_feat: [B, 16, H, W]
fused = torch.cat([s1_feat, s2_feat], dim=1) # Shape: [B, 32, H, W]
```

Spatial dimensions (H, W) and batch size (B) must match exactly.

3.4 The Core PyTorch Training Step

Every deep learning training step consists of six coordinated phases:

```
# 1. Zero out old gradients from the previous batch
optimizer.zero_grad()

# 2. Forward pass: compute predictions
```

```
pred = model(x_input)

# 3. Loss calculation: evaluate scalar error
loss = criterion(pred, target)

# 4. Backward pass: compute dLoss/dWeight for all parameters
loss.backward()

# 5. Gradient clipping: prevent exploding gradients
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

# 6. Optimizer step: update weights in the direction of negative gradient
optimizer.step()
```

3.5 Summary of Chapter 3

1. Neural network building blocks in PyTorch inherit from `torch.nn.Module`.
2. Layers and trainable weights are initialized in `__init__()`; execution logic resides in `forward()`.
3. Always call `model(x)` rather than `model.forward(x)` to preserve autograd hooks.
4. Broadcasting allows single-channel spatial masks to multiply multi-band optical tensors seamlessly.
5. The training cycle follows: `zero_grad()` → forward pass → loss calculation → `loss.backward()` → `optimizer.step()`.

3.6 Practice Exercises

1. What is the output shape when tensor **A** of shape $[2, 1, 64, 64]$ is multiplied element-wise with tensor **B** of shape $[2, 32, 64, 64]$?
2. What will happen during training if you forget to call `optimizer.zero_grad()` before `loss.backward()`?
3. Why does `torch.cat([tensor1, tensor2], dim=1)` require that dimensions 0, 2, and 3 match identically?

Solutions

1. Shape: $[2, 32, 64, 64]$. The singleton channel dimension of **A** is broadcast across the 32 channels of **B**.
2. Gradients in PyTorch accumulate by default (they are summed with existing values in `param.grad`). If `optimizer.zero_grad()` is omitted, gradients from the current batch will add to gradients from all preceding batches, causing the effective gradient magnitude to blow up and destabilize training.
3. Concatenation joins tensors along a specific axis without changing the geometric grid along any other axis. If height, width, or batch size differed, the elements could not form a valid rectangular multidimensional array.

Chapter 4

Neural Networks From Scratch

4.1 The Biological and Mathematical Artificial Neuron

An artificial neural network is a mathematical function that maps input vectors to output vectors through successive layers of linear transformations and non-linear activations.

The fundamental computational unit is the **artificial neuron**. Given an input vector $\mathbf{x} = [x_1, x_2, \ldots, x_n]^T$, the neuron computes a weighted sum, adds an adjustable scalar bias b, and passes the pre-activation through a non-linear activation function $\sigma(\cdot)$:

$$z = \sum_{i=1}^{n} w_i x_i + b = \mathbf{w}^T \mathbf{x} + b \tag{4.1}$$

$$a = \sigma(z) \tag{4.2}$$

- **Weights (w_i):** Control the sensitivity and importance of each input signal. A positive weight excites the neuron; a negative weight inhibits it.
- **Bias (b):** Shifts the activation threshold independently of the inputs, allowing the neuron to activate even when inputs are zero.
- **Activation Function (σ):** Introduces **non-linearity**. Without non-linear activations, stacking 100 neural network layers would mathematically collapse into a single matrix multiplication!

4.2 Essential Non-Linear Activation Functions

4.3 Forward and Backward Propagation: A Complete Numerical Example

Let us trace a concrete, end-to-end numerical calculation through a tiny artificial neuron to demystify how backpropagation and gradient descent work.

Mathematical Derivation & Numerical Example: Worked End-to-End Numerical Example

Initial Setup:

- Input value: $x = 2.0$
- True ground-truth target: $y^* = 1.0$
- Initial weight: $w = 0.5$

Function	Mathematical Formula	Output Range	Role in Satellite Vision
ReLU	$\max(0, z)$	$[0, \infty)$	Standard hidden layer activation; fast and avoids vanishing gradients for positive inputs.
LeakyReLU	$\max(\alpha z, z)$ $(\alpha = 0.2)$	$(-\infty, \infty)$	Used extensively in PatchGAN discriminators to prevent dead neurons when $z < 0$.
GELU	$z \cdot \Phi(z) \approx z \cdot \sigma(1.702z)$	$[-0.17, \infty)$	Smooth probabilistic activation used in modern vision transformers and ECRformer blocks.
Sigmoid	$\frac{1}{1+e^{-z}}$	$(0, 1)$	Compresses outputs into probabilities; ideal for soft cloud mask gates and attention weights.
Tanh	$\frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	$(-1, 1)$	Zero-centered non-linearity often used when normalizing optical ranges to $[-1, 1]$.

Table 4.1: Non-linear activation functions used across the ECRformer architecture and loss discriminators.

- Initial bias: $b = 0.0$
- Learning rate: $\eta = 0.1$
- Loss function: Mean Squared Error (MSE), $\mathcal{L} = \frac{1}{2}(a - y^*)^2$
- Activation: Linear identity for simplicity ($\sigma(z) = z$, so $a = z$).

Step 1: Forward Pass (Compute Prediction and Loss)

$$z = w \cdot x + b = (0.5)(2.0) + 0.0 = 1.0$$

$$a = \sigma(z) = 1.0$$

Suppose instead our initial weight was $w = 1.5$:

$$z = (1.5)(2.0) + 0.0 = 3.0 \implies a = 3.0$$

The error loss is:

$$\mathcal{L} = \frac{1}{2}(3.0 - 1.0)^2 = \frac{1}{2}(2.0)^2 = 2.0$$

Step 2: Backward Pass (Chain Rule Derivatives) We wish to know how much $\mathcal{L}$ changes when w changes: $\frac{\partial \mathcal{L}}{\partial w}$. By the calculus Chain Rule:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Evaluating each partial derivative:

1. Loss derivative with respect to output: $\frac{\partial \mathcal{L}}{\partial a} = (a - y^*) = (3.0 - 1.0) = +2.0$
2. Activation derivative: $\frac{\partial a}{\partial z} = 1.0$
3. Pre-activation derivative with respect to weight: $\frac{\partial z}{\partial w} = x = 2.0$

Multiplying them together:

$$\frac{\partial \mathcal{L}}{\partial w} = (+2.0) \times (1.0) \times (2.0) = +4.0$$

Similarly, for the bias b:

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial b} = (+2.0) \times (1.0) \times (1.0) = +2.0$$

Step 3: Parameter Update (Gradient Descent) We adjust parameters in the opposite direction of the gradient to reduce loss:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial \mathcal{L}}{\partial w} = 1.5 - (0.1)(+4.0) = 1.5 - 0.4 = 1.1$$

$$b_{\text{new}} = b_{\text{old}} - \eta \frac{\partial \mathcal{L}}{\partial b} = 0.0 - (0.1)(+2.0) = -0.2$$

Step 4: Verification (New Forward Pass) With updated parameters:

$$z_{\text{new}} = (1.1)(2.0) + (-0.2) = 2.2 - 0.2 = 2.0$$

$$\mathcal{L}_{\text{new}} = \frac{1}{2}(2.0 - 1.0)^2 = \frac{1}{2}(1.0)^2 = 0.5$$

The loss plummeted from $2.0 \rightarrow 0.5$ in a single gradient step!

4.4 From a Single Neuron to ECRformer

In ECRformer-Light, there are not 2 parameters, but **3,619,070** trainable parameters. However, the governing mathematical principle is identical:

1. High-dimensional tensors flow forward through millions of matrix operations.
2. A scalar composite loss measures the discrepancy against clear-sky Sentinel-2 targets.
3. PyTorch's automatic differentiation engine traverses the computational graph in reverse, calculating $\frac{\partial \mathcal{L}}{\partial \theta_i}$ for all 3.6 million parameters simultaneously using the Chain Rule.
4. The AdamW optimizer updates all weights to iteratively reduce cloud reconstruction artifacts.

4.5 Summary of Chapter 4

1. Artificial neurons compute affine transformations $\mathbf{w}^T\mathbf{x} + b$ followed by non-linear activations.
2. Without non-linear activations (such as ReLU, GELU, or LeakyReLU), multi-layer networks cannot model non-linear physical phenomena.
3. The Chain Rule enables calculating exact partial derivatives of the scalar loss with respect to every internal weight.
4. Gradient descent subtracts a fraction of the gradient ($\eta \nabla_\theta \mathcal{L}$) to drive errors downward.
5. Modern vision architectures scale this exact mechanism to millions of coupled parameters.

4.6 Practice Exercises

1. If a neuron has weight $w = 2.0$, bias $b = -1.0$, and receives input $x = 3.0$, what is the output a if the activation is ReLU(z)?

2. If the gradient $\frac{\partial \mathcal{L}}{\partial w} = -3.5$ and the learning rate is $\eta = 0.01$, does the weight increase or decrease after the optimizer step? Calculate the change Δw.

3. Why does the Sigmoid activation saturate (causing vanishing gradients) when input values $|z|$ are very large?

Solutions

1. $z = (2.0)(3.0) + (-1.0) = 6.0 - 1.0 = 5.0$. ReLU$(5.0) = \max(0, 5.0) = 5.0$.

2. The weight increases. $\Delta w = -\eta \frac{\partial \mathcal{L}}{\partial w} = -(0.01)(-3.5) = +0.035$.

3. As $z \to \pm\infty$, the Sigmoid curve becomes essentially horizontal ($\sigma(z) \to 1.0$ or 0.0). The derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ approaches 0.0, multiplying upstream gradients by near-zero and preventing preceding layers from learning.

Chapter 5

Convolutional Neural Networks and Image Features

5.1 Why Fully Connected Networks Fail on Satellite Imagery

If we flattened a 128×128 optical satellite patch with 4 channels into a 1D vector, it would contain $4 \times 128 \times 128 = 65{,}536$ numbers. Connecting this vector to a single hidden layer of 1,000 neurons using standard linear layers would require:

$$65{,}536 \times 1{,}000 \approx 65.5 \text{ million weights!}$$

Beyond consuming catastrophic amounts of memory, fully connected layers destroy the **2D spatial geometry** of the image: a pixel at coordinate $(0, 0)$ is treated as having no special geometric proximity to $(0, 1)$. Furthermore, if an agricultural parcel or cloud boundary shifts three pixels to the right, a fully connected network fails to recognize it.

5.2 The Convolution Operation

Convolutional Neural Networks (CNNs) solve this problem through two core inductive biases:

1. **Local Receptive Fields:** Each neuron connects only to a small spatial neighborhood (e.g., 3×3 or 7×7 pixels).
2. **Weight Sharing (Translation Equivariance):** The same small matrix of weights (a **filter** or **kernel**) is slid (convolved) across the entire image grid. An edge detector learned in the top-left corner automatically detects edges in the bottom-right corner!

Mathematical Derivation & Numerical Example: A Complete Numerical 2D Convolution Example

Consider a 4×4 single-channel input matrix $\mathbf{X}$ and a 3×3 horizontal Sobel edge-detection kernel $\mathbf{K}$ (with no padding and stride $s = 1$):

$$\mathbf{X} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Notice the sharp horizontal edge between row 2 and row 3 in $\mathbf{X}$. The output feature map

Y has spatial dimensions:

$$H_{\text{out}} = \frac{H - K + 2P}{S} + 1 = \frac{4 - 3 + 0}{1} + 1 = 2$$

Computing the top-left output element $\mathbf{Y}_{0,0}$ (overlapping rows 0–2, cols 0–2 of **X**):

$$\begin{aligned}\mathbf{Y}_{0,0} &= (1)(-1) + (1)(-2) + (1)(-1) \\ &\quad + (1)(0) + (1)(0) + (1)(0) \\ &\quad + (0)(1) + (0)(2) + (0)(1) \\ &= -1 - 2 - 1 + 0 + 0 = \mathbf{-4}\end{aligned}$$

Computing the bottom-left output element $\mathbf{Y}_{1,0}$ (overlapping rows 1–3, cols 0–2 of **X**):

$$\begin{aligned}\mathbf{Y}_{1,0} &= (1)(-1) + (1)(-2) + (1)(-1) \\ &\quad + (0)(0) + (0)(0) + (0)(0) \\ &\quad + (0)(1) + (0)(2) + (0)(1) \\ &= -4\end{aligned}$$

The kernel responds with maximum magnitude (-4) precisely along the boundary between bright and dark regions, extracting an explicit edge feature map!

5.3 Padding and the Importance of Reflection Padding

When convolving with a 3×3 kernel, the outer border pixels have fewer neighbors. Standard zero-padding appends zeros around the perimeter. However, in satellite imagery:

- Appending zeros creates artificial black cliffs at the image boundaries.
- In cloud reconstruction, zero-padding causes dark framing artifacts along the perimeter of the reconstructed patch.

To eliminate boundary distortion, our implementation uses **reflection padding**:

$$[\ldots, x_2, x_1, x_0 \mid x_0, x_1, x_2, \ldots] \tag{5.1}$$

The boundary pixels are mirrored outward, ensuring continuity for gradient filters and transformer attention windows.

Code Implementation Traceability: Reflection Padding in `models/ecrformer_model.py`

Notice how the decoupled input stem and final reconstruction head in our codebase explicitly enforce reflection padding:

```
# In DecoupledEncoder (models/ecrformer_model.py, line 363)
self.branch_0 = nn.Conv2d(in_chans[0], features_start // 2,
                          kernel_size=7, padding=3, padding_mode='
                              reflect')

# In ECRformerAdaptedModel (models/ecrformer_adapted.py, line 58)
self.final_conv = nn.Conv2d(feats[0], out_chans,
                            kernel_size=3, padding=1, padding_mode='
```

```
                                    reflect')
```

5.4 Modern Upsampling: PixelShuffle vs. Transposed Convolution

Early generative models used `nn.ConvTranspose2d` (deconvolution) to upsample feature maps. However, transposed convolutions with stride 2 suffer from uneven kernel overlaps, producing notorious **checkerboard artifacts** across the image.

ECRformer-Light eliminates checkerboard artifacts by utilizing **PixelShuffle** (sub-pixel convolution, Shi et al. [8]) and its inverse, **PixelUnshuffle**:

- **PixelUnshuffle ($r = 2$):** Takes a tensor $[B, C, H, W]$ and rearranges non-overlapping 2×2 spatial patches into the channel dimension, producing $[B, 4C, H/2, W/2]$. It downsamples spatial resolution with **zero loss of information**!
- **PixelShuffle ($r = 2$):** Takes a tensor $[B, 4C, H, W]$ and rearranges 4 channel elements into a 2×2 spatial grid, producing $[B, C, 2H, 2W]$. It produces smooth, mathematically continuous upsampling.

Code Implementation Traceability: Downsamplers and Upsamplers in `models/ecrformer_model.py`

```
# Downsampler: Conv reduces channels, PixelUnshuffle doubles
    resolution losslessly
self.downsamplers.append(nn.Sequential(
    nn.Conv2d(in_ch, out_ch // 4, kernel_size=3, padding=1),
    nn.PixelUnshuffle(2), # Expands (out_ch // 4) * 4 -> out_ch
    CBAM(out_ch)
))

# Upsampler: Conv expands channels by 4x, PixelShuffle upsamples 2x
    spatially
self.upsamplers.append(nn.Sequential(
    nn.Conv2d(in_ch, out_ch * 4, kernel_size=3, padding=1),
    nn.PixelShuffle(2),  # (out_ch * 4) -> out_ch with 2x spatial
        height/width
    CBAM(out_ch)
))
```

5.5 Summary of Chapter 5

1. CNNs exploit local spatial coherence and translation equivariance, making them vastly more parameter-efficient than dense linear layers.
2. Convolving learnable kernels over feature maps extracts low-level edges, mid-level textures, and high-level semantics.
3. Reflection padding mirrors edge pixels, eliminating the black-border artifacts caused by standard zero-padding.
4. PixelUnshuffle and PixelShuffle provide lossless spatial downsampling and artifact-free upsampling, completely bypassing checkerboard artifacts.

5.6 Practice Exercises

1. Given an input of shape $[B, 32, 64, 64]$, what is the exact output shape after applying `nn.PixelUnshuffle(2)`?

2. Why does zero-padding create undesirable artifacts in satellite image reconstruction?

3. How does a 7×7 convolution in the input stem differ in receptive field compared to a standard 3×3 convolution?

Solutions

1. Spatial dimensions are halved ($64/2 = 32$) and channels are multiplied by $2^2 = 4$ ($32 \times 4 = 128$). Output shape: $[B, 128, 32, 32]$.

2. Zero-padding fills borders with numerical zeros. In reflectance terms, 0.0 represents absolute total absorption (pure black), creating artificial step-function gradients along the patch borders that distort optical edge features.

3. A 7×7 kernel directly inspects a 70 m $\times$ 70 m spatial neighborhood on the ground in a single step, enabling the input stem to capture broader macroscopic geological and agricultural structures immediately. A 3×3 kernel only observes 30 m $\times$ 30 m.

Chapter 6

The Complete Data Pipeline

6.1 End-to-End Architectural Overview

A deep learning system is far more than its neural network weights: the data pipeline dictates what physical information reaches the model, how it is normalized, how gradients are calculated, and how final predictions are assembled.

Figure 6.1 illustrates the verified data and training pipeline running across our repository.

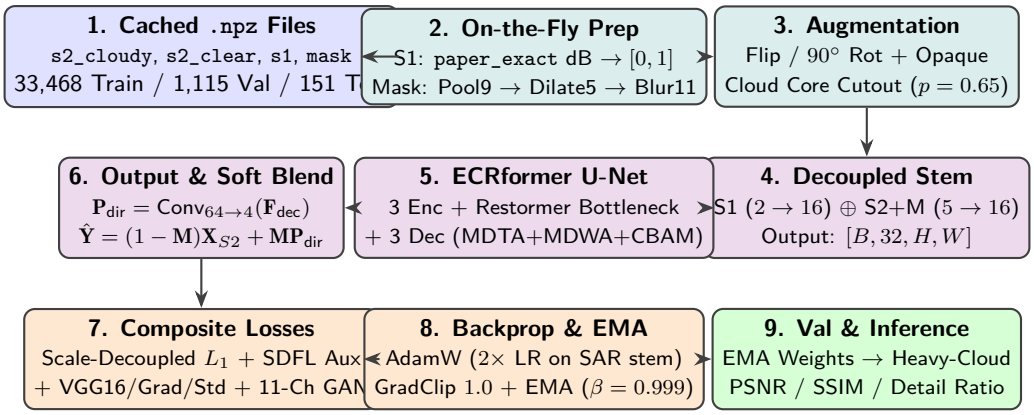


Figure 6.1: Complete end-to-end training, validation, and inference workflow verified across `data/cache_dataset.py`, `models/ecrformer_adapted.py`, and `train_ecrformer_v4_gan.py`.

6.2 Step-by-Step Trajectory of a Satellite Patch

Let us follow a single geographical coordinate through the entire system from disk storage to clear-sky optical reconstruction:

6.3 Failure Modes in Multimodal Data Pipelines

Understanding where pipelines fail is vital for diagnosing poor model performance:

- **Data Leakage via Scene Overlap:** If adjacent 128×128 patches from the same geographical orbit are randomly split between training and validation sets, the model simply memorizes the local terrain. Our pipeline enforces strict **scene-level splitting** (splitting by ROI scene directory), guaranteeing zero spatial overlap between training and testing.

Stage	Source File & Function	Tensor Shape	Physical Operation
1. Raw Cache	`data/cache_dataset.py`	Disk `.npz` files	Loads compressed arrays: `s2_cloudy`, `s2_clear`, `s1`, `mask`.
2. SAR Norm	`CachedSEN12MSCRDataset`	S1: $[2, 128, 128]$	Clamps VV $\in [-25, 0]$ dB, VH $\in [-35, 0]$ dB; scales linearly to $[0, 1]$.
3. Mask Prep	`__getitem__` lines 84–97	Mask: $[1, 128, 128]$	3-step refinement: 9×9 pool $\rightarrow 5 \times 5$ dilate $\rightarrow$ 11×11 Gaussian blur.
4. Augment	`apply_cloud_cutout`	Opt: $[4, 128, 128]$	Random flips/rotations; synthetic opaque cutout in cloud cores ($p = 0.65$).
5. Stacking	`torch.cat`	$\mathbf{X}_{\text{in}} : [B, 7, 128, 128]$	Concatenates S1 (2 ch), S2 cloudy (4 ch), and soft mask (1 ch).
6. Decoupled Stem	`DecoupledEncoder`	$\mathbf{F}_0 : [B, 32, 128, 128]$	S1 $\rightarrow$ $\text{Conv}_{2\rightarrow16}$; S2+M $\rightarrow$ $\text{Conv}_{5\rightarrow16}$; concatenates to 32 ch.
7. 4-Stage U-Net	`ECRformerModel`	$[B, 256, 16, 16] \rightarrow [B, 64, 128, 128]$	Multi-scale MDTA + MDWA attention blocks with CBAM channel/spatial attention.
8. Final Conv	`final_conv`	$\mathbf{P}_{\text{dir}} : [B, 4, 128, 128]$	3×3 reflection conv initialized with Sentinel-2 global clear channel means.
9. Soft Blending	`ECRformerAdaptedModel`	$\hat{\mathbf{Y}} : [B, 4, 128, 128]$	$\hat{\mathbf{Y}} = (1 - \mathbf{M}) \odot \mathbf{X}_{S2} + \mathbf{M} \odot \mathbf{P}_{\text{dir}}$.
10. Supervision	`train_ecrformer_v4_gan.py`	Scalar loss $\mathcal{L}_{\text{total}}$	Scale-decoupled L_1 + SDFL aux + VGG/-Grad/Std + 11-Ch PatchGAN.
11. Optimizer	AdamW + GradClip	Model weights θ	$2\times$ learning rate on SAR branch; EMA update ($\beta = 0.999$).

Table 6.1: Verified execution stages across the entire training and inference pipeline.

- **SAR Decibel Clipping Mismatch:** If the training script clips SAR VV to $[-25, 0]$ dB but inference clips to $[-30, 0]$ dB, the entire calibration shift blinds the model's radar feature extractors.
- **Inverted Cloud Mask:** If the mask defines 1 = clear and 0 = cloudy, the soft-blend equation $(1-\mathbf{M})\mathbf{X}_{S2}+\mathbf{M}\mathbf{P}_{\text{dir}}$ will copy the cloudy pixels directly into the final image while replacing clear pixels with neural predictions!

6.4 Summary of Chapter 6

1. The pipeline ingests cached multi-sensor arrays and executes deterministic, on-the-fly radiometric normalizations.
2. Sentinel-1 radar and Sentinel-2 optical data are processed through separate decoupled convolutional branches before fusion.
3. Spatial feature maps traverse a 4-level encoder-decoder hierarchy equipped with transposed and dilated window attention.
4. Soft mask blending guarantees that pristine clear-sky optical pixels are preserved with 100% mathematical fidelity.
5. Gradient updates employ differential learning rates, gradient norm clipping, and exponential moving average (EMA) weight accumulation.

6.5 Practice Exercises

1. Why must the dataset be split at the *scene level* rather than by shuffling individual patches?
2. If a pixel in the cloud mask has a value of $\mathbf{M} = 0.0$, what is the exact formula for the blended output pixel $\hat{\mathbf{Y}}$?
3. What is the role of Exponential Moving Average (EMA) during model checkpointing?

Solutions

1. Adjacent 128×128 patches cut from the same satellite acquisition share identical atmospheric conditions, soil types, and seasonal vegetation phenology. Random patch shuffling allows the model to memorize the landscape during training, artificially inflating validation scores without proving generalizability to unseen regions. Scene-level splitting tests the model on completely different geographic regions.
2. $\hat{\mathbf{Y}} = (1-0.0)\mathbf{X}_{S2}+(0.0)\mathbf{P}_{\text{dir}} = \mathbf{X}_{S2}$. The output is mathematically identical to the original Sentinel-2 optical pixel, introducing zero neural distortion or blurring to clear areas.
3. EMA maintains a running temporal average of the model weights: $\theta_{\text{EMA}} \leftarrow \beta\theta_{\text{EMA}} + (1-\beta)\theta$. This filters out high-frequency stochastic gradient noise from individual mini-batches, consistently producing smoother convergence and higher PSNR/SSIM on validation benchmarks.

Chapter 7

SEN12MS-CR Dataset and Preprocessing

7.1 The SEN12MS-CR Benchmark Dataset

The SEN12MS-CR (Sentinel-1 and Sentinel-2 Multi-Spectral Dataset for Cloud Removal) dataset, introduced by Ebel et al. [1], is the global benchmark for multimodal satellite image reconstruction.

The dataset spans all meteorological seasons and continents, covering diverse biomes including dense Amazonian rainforests, arid deserts, European agricultural farmlands, urban megacities, and Arctic tundras.

7.1.1 Structure of a Multimodal Triplet

Each observation consists of three spatially co-registered data arrays acquired over the exact same geographical bounding box:

1. **Input Sentinel-2 Optical Image ($\mathbf{X_{cloudy}}$):** Acquired on a cloudy day, containing 4 spectral bands (`B04`, `B03`, `B02`, `B08`) contaminated by cumulus, stratus, cirrus, or haze.
2. **Input Sentinel-1 SAR Image ($\mathbf{X_{SAR}}$):** Acquired close in time to the cloudy optical acquisition, containing VV and VH microwave backscatter channels.
3. **Ground-Truth Reference Optical Image ($\mathbf{Y^*_{clear}}$):** Acquired over the same scene on a nearby clear-sky date.

Common Pitfall & Caution: The Reality of Satellite Ground-Truth: Temporal Offset

In satellite remote sensing, **it is physically impossible to acquire a cloud-free optical image and a cloudy optical image at the exact same second in time!** The clear reference target is taken from a different satellite overpass (typically a few days or weeks apart). As a consequence:

- Seasonal crop harvesting or plowing may have occurred between the two dates.
- Soil moisture or river water levels may have changed.
- The reference image may occasionally contain faint residual cirrus haze or cloud shadows.

A research-level engineer must recognize that the ground-truth target contains natural temporal variations, meaning that penalizing tiny high-frequency pixel differences too harshly can cause the network to overfit to temporal noise!

7.2 Dataset Splitting: Eliminating Spatial Leakage

Our dataset is pre-processed and cached in the directory `cache_fall_v3` across three partitions:

- **Training Set (`train`):** 33,468 patches.
- **Validation Set (`val`):** 1,115 patches.
- **Test Set (`test`):** 151 patches.

Crucially, these splits are partitioned by **Region of Interest (ROI) scene directory** rather than by individual patches. If patches from the same 100 km × 100 km tile were randomly mixed, the model could easily memorize road networks and geographic coastlines, creating the illusion of generalization. Scene-level splitting guarantees that the validation and test patches come from geographical territories that the model has never encountered during training!

7.3 Exact Radiometric Normalization Formulas

Raw sensor readings have vastly different physical units and dynamic ranges. Feeding unnormalized values into a neural network destabilizes activation variances and causes vanishing or exploding gradients.

7.3.1 Optical Reflectance Normalization

Raw Sentinel-2 Level-1C and Level-2A data are distributed as 16-bit integer Digital Numbers ($\text{DN} \in [0, 10000]$). In our cached dataset:

$$\mathbf{X}_{\text{opt}} = \text{clip}\left(\frac{\text{DN}}{10000.0}, 0.0, 1.0\right) \tag{7.1}$$

A value of 0.0 corresponds to 0% surface reflectance (absolute absorption), while 1.0 corresponds to 100% surface reflectance. In practice, dense clouds often saturate near 0.8–1.0, while typical clear vegetation has reflectances of 0.03–0.15 in visible bands and 0.30–0.50 in the NIR band.

7.3.2 Sentinel-1 SAR Normalization: `paper_exact`

Synthetic Aperture Radar backscatter is measured in logarithmic **decibels (dB)**:

$$\sigma^0_{\text{dB}} = 10 \log_{10}(\sigma^0) \tag{7.2}$$

Typical terrestrial SAR backscatter values range from -35 dB (smooth calm water or tarmac) to 0 dB (corner reflectors in urban buildings). In `data/cache_dataset.py`, our active training pipeline applies the `paper_exact` normalization:

$$\mathbf{X}_{\text{VV, norm}} = \frac{\text{clip}(\sigma^0_{\text{VV}}, -25.0, 0.0) + 25.0}{25.0} \in [0, 1] \tag{7.3}$$

$$\mathbf{X}_{\text{VH, norm}} = \frac{\text{clip}(\sigma^0_{\text{VH}}, -35.0, 0.0) + 35.0}{35.0} \in [0, 1] \tag{7.4}$$

Mathematical Derivation & Numerical Example: SAR Normalization Walkthrough

Suppose a patch contains two pixels: Pixel A over a calm lake and Pixel B over an urban skyscraper.

1. **Pixel A (Calm Lake):**

$$\sigma^0_{\text{VV}} = -28.4 \text{ dB}, \quad \sigma^0_{\text{VH}} = -38.1 \text{ dB}$$

Clamping: $\sigma^0_{\text{VV}} \to -25.0$, $\sigma^0_{\text{VH}} \to -35.0$. Normalized values:

$$\mathbf{X}_{\text{VV, norm}} = \frac{-25.0 + 25.0}{25.0} = \mathbf{0.0}, \quad \mathbf{X}_{\text{VH, norm}} = \frac{-35.0 + 35.0}{35.0} = \mathbf{0.0}$$

2. **Pixel B (Urban Skyscraper):**

$$\sigma^0_{\text{VV}} = -5.0 \text{ dB}, \quad \sigma^0_{\text{VH}} = -14.0 \text{ dB}$$

Clamping: Values are already within $[-25, 0]$ and $[-35, 0]$. Normalized values:

$$\mathbf{X}_{\text{VV, norm}} = \frac{-5.0 + 25.0}{25.0} = \frac{20.0}{25.0} = \mathbf{0.80}$$

$$\mathbf{X}_{\text{VH, norm}} = \frac{-14.0 + 35.0}{35.0} = \frac{21.0}{35.0} = \mathbf{0.60}$$

Both radar and optical channels are now bounded within $[0, 1]$, enabling stable multi-modal feature fusion.

Code Implementation Traceability: Code Implementation in `data/cache_dataset.py`

```
# Lines 63-75 in data/cache_dataset.py
if self.sar_norm == 'paper_exact':
    # VV: [-25, 0] dB -> [0, 1]
    # VH: [-35, 0] dB -> [0, 1]
    s1[0] = np.clip(s1[0], -25.0, 0.0)
    s1[0] = (s1[0] + 25.0) / 25.0
    s1[1] = np.clip(s1[1], -35.0, 0.0)
    s1[1] = (s1[1] + 35.0) / 35.0
```

7.4 Summary of Chapter 7

1. SEN12MS-CR provides spatially co-registered triplets of cloudy optical, SAR, and clear-sky reference imagery across global biomes.
2. Due to orbital revisit schedules, clear-sky ground-truth imagery is acquired on a nearby date, introducing subtle temporal phenology variations.
3. Strict scene-level dataset splitting prevents spatial data leakage between training, validation, and test sets.
4. Optical surface reflectance is normalized to $[0, 1]$ by dividing digital numbers by 10,000.
5. Sentinel-1 SAR dB backscatter is clipped to $[-25, 0]$ dB (VV) and $[-35, 0]$ dB (VH) and scaled to $[0, 1]$.

7.5 Practice Exercises

1. If a radar backscatter reading for VH is recorded as -42.0 dB, what is its normalized value under the `paper_exact` scheme?

2. Why are different clipping thresholds used for Sentinel-1 VV (-25 dB) and VH (-35 dB)?

3. A raw Sentinel-2 pixel in band `B08` has a Digital Number of 3,450. What is its surface reflectance ρ?

Solutions

1. Clamping -42.0 dB to $[-35.0, 0.0]$ yields -35.0 dB. Normalized: $\frac{-35.0+35.0}{35.0} = 0.0$.

2. VH is a cross-polarized signal (vertical transmit, horizontal receive). Due to physical polarization depolarization losses in microwave backscattering, VH return signals are naturally 6–10 dB weaker than co-polarized VV returns. Lowering the minimum threshold to -35 dB preserves subtle vegetation volume scattering that would otherwise be clipped out.

3. $\rho = \frac{3450}{10000.0} = 0.345$ (characteristic of moderate green vegetation canopy reflectance in NIR).

Chapter 8

Cloud Masks and Mask-Guided Learning

8.1 What is a Cloud Mask?

A cloud mask is a single-channel spatial map $\mathbf{M} \in \mathbb{R}^{1\times H\times W}$ that indicates the spatial presence and optical density of clouds across an image patch:

- $\mathbf{M}_{h,w} = 0.0$: The pixel is pristine, clear-sky ground surface.
- $\mathbf{M}_{h,w} = 1.0$: The pixel is covered by an opaque cloud core.
- $0.0 < \mathbf{M}_{h,w} < 1.0$: The pixel is covered by semi-transparent cloud boundaries, cirrus streamers, or atmospheric haze.

8.2 The Three-Step Mask Refinement Pipeline

In our dataset cache, raw binary masks were initially detected using the gradient-boosted decision tree algorithm `s2cloudless`. However, raw detector masks frequently contain high-frequency false positives (such as bright sandy beaches or white buildings mistaken for clouds) and sharp, jagged cloud borders.

In `data/cache_dataset.py` (lines 84–97), our dataset loader applies an automated **3-step morphological and spatial refinement**:

1. **Speckle Noise Removal (9×9 Average Pooling):**

$$\mathbf{M}_{\text{clean}} = \mathbb{I}\left(\text{AvgPool}_{9\times 9}(\mathbf{M}_{\text{raw}}) > 0.5\right) \tag{8.1}$$

   Isolated 1-pixel false detections vanish because their local 9×9 average is far below 0.5.

2. **Morphological Dilation (5×5 Max Pooling):**

$$\mathbf{M}_{\text{dilated}} = \text{MaxPool}_{5\times 5}(\mathbf{M}_{\text{clean}}) \tag{8.2}$$

   Clouds cast faint shadows and diffuse penumbra haze around their edges. Dilating the mask by a 2-pixel radius (5×5 kernel) ensures that faint transitional haze surrounding thick clouds is encompassed by the mask.

3. **Gaussian Softening (11×11, $\sigma = 2.0$):**

$$\mathbf{M}_{\text{soft}} = \text{clip}\left(\mathbf{G}_{\sigma=2.0} * \mathbf{M}_{\text{dilated}}, 0.0, 1.0\right) \tag{8.3}$$

   This converts hard binary step edges ($0 \to 1$) into a smooth, continuous mathematical sigmoid-like transition over a width of $\approx$ 5–8 pixels.

Common Pitfall & Caution: Why Binary Masks Cause Border Seams

If a binary mask ($\mathbf{M} \in \{0, 1\}$) is used to stitch a neural network prediction into a clear image, a sharp mathematical cliff occurs at the boundary: pixel (x, y) is pure original optical data, while neighbor $(x + 1, y)$ is pure neural network prediction. Because lighting and atmospheric scattering differ slightly between the two dates, an unmistakable, artificial jagged seam is visible to human observers. The 11×11 Gaussian blur creates an imperceptible alpha-blend transition that eliminates seams completely!

8.3 The Three Critical Mask Roles: Never Confuse Them!

One of the most frequent misconceptions in deep learning satellite research is conflating the different ways a mask is used. Our codebase employs the mask in **three completely distinct roles**:

Mask Role	Mathematical Equation	System Purpose
1. Input Conditioning	$\mathbf{X}_{\text{in}} = [\mathbf{X}_{\text{SAR}}, \mathbf{X}_{S2}, \mathbf{M}] \in \mathbb{R}^{7\times H\times W}$	Informs the encoder where optical data is reliable vs. corrupt, allowing attention layers to dynamically route features.
2. Loss Weighting	$\mathcal{L}_{\text{cloud}} = \frac{\sum(\mathbf{M}\odot\lvert\hat{\mathbf{Y}}-\mathbf{Y}^*\rvert)}{\sum\mathbf{M}+\epsilon}$	Directs backpropagation gradients to penalize reconstruction errors inside cloudy regions while ignoring clear terrain.
3. Output Soft Blending	$\hat{\mathbf{Y}} = (1-\mathbf{M})\odot\mathbf{X}_{S2}+\mathbf{M}\odot\mathbf{P}_{\text{dir}}$	Guarantees that clear-sky optical pixels are preserved with 100% original fidelity and zero neural distortion.

Table 8.1: The three distinct roles of the cloud mask in ECRformer-Light.

Mathematical Derivation & Numerical Example: A 4×4 Soft-Blend Worked Example

Let us trace the soft-blending equation across a 1D slice of 4 adjacent pixels transitioning from clear sky into a thick cloud:

$$\mathbf{M} = \begin{bmatrix}0.00 & 0.25 & 0.75 & 1.00\end{bmatrix} \quad \text{(Gaussian softened mask)}$$

Suppose the cloudy optical input is:

$$\mathbf{X}_{S2} = \begin{bmatrix}0.12 & 0.35 & 0.85 & 0.98\end{bmatrix} \quad \text{(clear terrain } \rightarrow \text{ bright cloud)}$$

And the neural network's direct prediction $\mathbf{P}_{\text{dir}}$ reconstructed from radar is:

$$\mathbf{P}_{\text{dir}} = \begin{bmatrix}0.14 & 0.15 & 0.13 & 0.14\end{bmatrix} \quad \text{(reconstructed underlying terrain)}$$

Evaluating the blended output $\hat{\mathbf{Y}} = (1 - \mathbf{M})\mathbf{X}_{S2} + \mathbf{M}\mathbf{P}_{\text{dir}}$:

$\hat{\mathbf{Y}}_0 = (1 - 0.00)(0.12) + (0.00)(0.14) = \mathbf{0.120}$ (100% original ground truth preserved!)

$\hat{\mathbf{Y}}_1 = (1 - 0.25)(0.35) + (0.25)(0.15) = 0.2625 + 0.0375 = \mathbf{0.300}$

$\hat{\mathbf{Y}}_2 = (1 - 0.75)(0.85) + (0.75)(0.13) = 0.2125 + 0.0975 = \mathbf{0.310}$

$\hat{\mathbf{Y}}_3 = (1 - 1.00)(0.98) + (1.00)(0.14) = 0.000 + 0.140 = \mathbf{0.140}$ (100% neural reconstruction)

The transition is perfectly smooth and physically grounded!

8.4 Summary of Chapter 8

1. Cloud masks map spatial cloud probability, where 0.0 = clear and 1.0 = cloudy.
2. A 3-step refinement pipeline (average pooling, max-pool dilation, Gaussian softening) eliminates speckle noise and hard boundary cliffs.
3. Input conditioning feeds the mask as channel 6, allowing self-attention to gauge optical reliability.
4. Loss weighting concentrates gradient updates on cloudy pixels.
5. Output soft blending guarantees zero modification of clear-sky pixels while seamlessly transitioning into reconstructed regions.

8.5 Practice Exercises

1. Why does applying a 5×5 max-pooling operation perform morphological dilation on a binary mask?
2. If the cloud mask has value $\mathbf{M} = 1.0$ at pixel (i, j), how much does the original cloudy optical value $\mathbf{X}_{S2}(i, j)$ contribute to the final reconstructed output $\hat{\mathbf{Y}}(i, j)$?
3. What visual defect occurs if an unblurred binary mask is used for final output blending?

Solutions

1. Max-pooling returns the maximum value in every 5×5 window. If even a single pixel in the window has value 1.0 (cloud), the center pixel becomes 1.0. This mathematically expands (dilates) the perimeter of cloud regions outward by $\lfloor 5/2 \rfloor = 2$ pixels.
2. Zero (0%). When $\mathbf{M} = 1.0$, the weight on $\mathbf{X}_{S2}$ is $(1 - \mathbf{M}) = (1 - 1.0) = 0.0$. The output is 100% derived from the neural network's direct prediction $\mathbf{P}_{\text{dir}}$.
3. An artificial, razor-sharp perimeter seam or "cookie-cutter" border appears around cloud edges, creating noticeable contrast discontinuities against surrounding clear terrain.

Chapter 9

Encoder-Decoder Architecture

9.1 The Philosophy of Encoder-Decoder Networks

Satellite image cloud removal requires solving two opposing objectives simultaneously:

1. **Global Semantic Understanding:** The network must understand macroscopic environmental context (e.g., recognizing that an entire 10 km valley is agricultural farmland or an urban industrial zone).
2. **Precise High-Frequency Localization:** The network must reconstruct razor-sharp river boundaries, highway lanes, and field borders at the individual 10-meter pixel level.

To balance these goals, modern vision systems utilize an **Encoder-Decoder U-Net** architecture (Ronneberger et al. [7]).

Concept: Analogy: The Architect and the Construction Crew

- **The Encoder (The Architect):** Gradually compresses high-resolution spatial details into abstract, low-resolution semantic concepts (e.g., "this region is a pine forest adjacent to a reservoir"). It sacrifices fine pixel coordinates to understand the "big picture."
- **The Bottleneck (The Blueprint):** The most compressed, abstract summary of the scene.
- **The Decoder (The Construction Crew):** Gradually expands the abstract blueprint back up to full image resolution.
- **Skip Connections (Direct Telephone Lines):** Direct communication channels routing uncompressed high-resolution spatial features from each encoder level directly across to the corresponding decoder level!

9.2 The Complete ECRformer-Light U-Net Hierarchy

Figure 9.1 diagrams the exact, verified tensor dimensions and channel pathways in `ECRformerAdaptedModel`.

9.2.1 Stage-by-Stage Spatial and Channel Evolution

Tracing a 128×128 input through each stage:

Stage	Module Name	Spatial Grid	Channels	Internal Architecture
Input	Raw Input Tensor	128×128	7	S1 (2 ch) $\oplus$ S2 (4 ch) $\oplus$ Mask (1 ch)
Stem	`DecoupledEncoder`	128×128	32	$\text{Conv}_{2\to16}(k = 7) \oplus \text{Conv}_{5\to16}(k = 7)$
Enc 0	`encoder[0]`	128×128	32	$2\times$ ECRformerBlock + CBAM ($C = 32$)
Down 0	`downsamplers[0]`	64×64	64	$\text{Conv}_{32\to16}$ + PixelUnshuffle(2) + CBAM
Enc 1	`encoder[1]`	64×64	64	$2\times$ ECRformerBlock + CBAM ($C = 64$)
Down 1	`downsamplers[1]`	32×32	128	$\text{Conv}_{64\to32}$ + PixelUnshuffle(2) + CBAM
Enc 2	`encoder[2]`	32×32	128	$1\times$ ECRformerBlock + CBAM ($C = 128$)
Down 2	`downsamplers[2]`	16×16	256	$\text{Conv}_{128\to64}$ + PixelUnshuffle(2) + CBAM
Bottleneck	`bottle_neck_module`	16×16	256	$1\times$ RestormerBlock (MDTA + GDFN)
Up 0	`upsamplers[0]`	32×32	128	$\text{Conv}_{256\to512}$ + PixelShuffle(2) + CBAM
Dec 0	`decoder[0]`	32×32	128	$\text{Conv}_{256\to128}$ + $1\times$ ECRformerBlock + CBAM
Up 1	`upsamplers[1]`	64×64	64	$\text{Conv}_{128\to256}$ + PixelShuffle(2) + CBAM
Dec 1	`decoder[1]`	64×64	64	$\text{Conv}_{128\to64}$ + $2\times$ ECRformerBlock + CBAM
Up 2	`upsamplers[2]`	128×128	32	$\text{Conv}_{64\to128}$ + PixelShuffle(2) + CBAM
Dec 2	`decoder[2]`	128×128	64	Identity Fuser + $2\times$ ECRformerBlock + CBAM
Head	`final_conv`	128×128	4	$\text{Conv}_{64\to4}(k = 3, p = 1, \text{reflect})$

Table 9.1: Exact layer progression and tensor geometries inside `ECRformerAdaptedModel`.

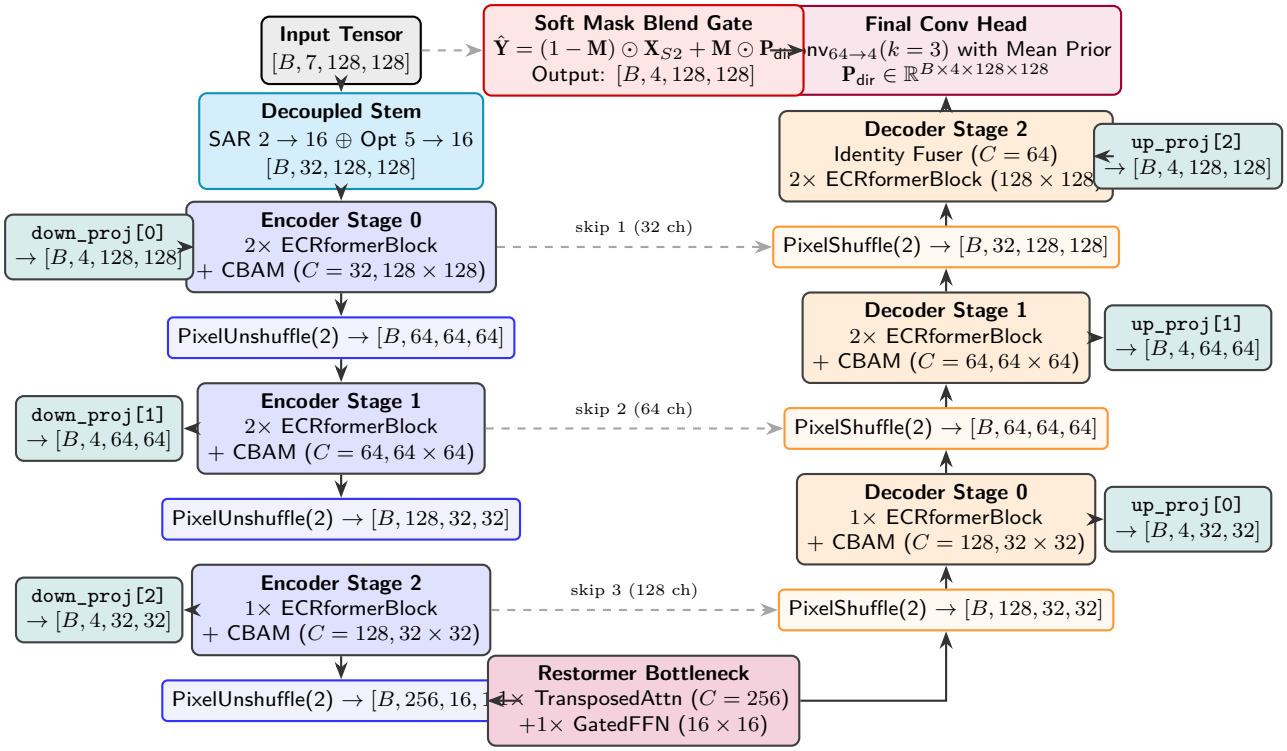


Figure 9.1: Verified complete architecture of `ECRformerAdaptedModel`. Exact spatial resolutions and channel counts at each encoder and decoder level are annotated. The auxiliary projection heads (`down_proj[0..2]` and `up_proj[0..2]`) provide multi-scale supervision during training.

Research-Level Deep Dive: Resolving the 3.6M vs. 8.3M Parameter Contradiction

In previous documentation and metadata files, the model size was reported as **8,316,228 parameters**. Our direct code and checkpoint tensor audit revealed the exact breakdown:

- **Active Generator (`model_state_dict`):** Exactly **3,619,070 trainable parameters** across 277 tensors!
- **Exponential Moving Average Shadow Weights (`ema_state_dict`):** Exactly **3,619,070 parameters** (an exact mirror copy of the generator weights).
- **PatchGAN Discriminator (`disc_state_dict`):** Exactly **1,260,737 trainable parameters** (1,269,298 tensor elements including spectral norm buffers).

Summing all three sub-dictionaries stored inside the single checkpoint file on disk yields:

$$3{,}619{,}070 + 3{,}619{,}070 + 1{,}269{,}298 = \mathbf{8{,}507{,}438} \text{ total tensor floats} \approx 34.3 \text{ MB}$$

The original author counted all saved tensors in the checkpoint file together! The active generator running inference in memory is a lean, highly efficient **3.62 million parameter** model!

9.3 Summary of Chapter 9

1. Encoder-decoder networks compress input imagery to capture global context before reconstructing high-resolution outputs.
2. Skip connections concatenate uncompressed encoder feature maps with upsampled decoder features, restoring high-frequency textures.
3. ECRformer-Light utilizes a 4-level hierarchy downsampling from 128×128 to 16×16 at the bottleneck ($C = 256$).
4. PixelUnshuffle and PixelShuffle provide lossless spatial resampling without checkerboard artifacts.
5. The active generator contains 3,619,070 parameters; the 8.3M figure in previous reports resulted from summing the generator, EMA shadow weights, and discriminator stored in the checkpoint file.

9.4 Practice Exercises

1. Why does `decoder[0]` receive a 256-channel input before its fuser convolution reduces it to 128 channels?
2. If skip connections were completely removed from the U-Net, what visual defect would appear in the reconstructed satellite images?
3. How many times is the spatial resolution reduced between the input patch and the Restormer bottleneck?

Solutions

1. `upsampler[0]` produces a 128-channel feature map. It is concatenated along $C = 1$ with the 128-channel skip connection coming from `encoder[2]` ($128 + 128 = 256$ channels). The fuser convolution (`Conv2d(256, 128)`) then combines both feature streams back into 128 channels.
2. Without skip connections, all fine spatial coordinate information must pass through the heavily compressed 16×16 bottleneck. The resulting output images would look excessively blurry, with smudged coastlines, distorted roads, and washed-out textures.
3. The spatial dimensions are halved three times: $128 \to 64 \to 32 \to 16$. This represents an $8\times$ spatial downsampling along each axis ($64\times$ reduction in spatial grid area).

Chapter 10

Transformers and Attention Mechanisms From Scratch

10.1 The Fundamental Limitation of Convolutional Networks

In a standard convolutional layer, each output neuron only sees a small local patch (e.g., 3×3 pixels). To capture relationships between two pixels separated by 80 pixels, a CNN must stack dozens of layers so that their receptive fields gradually overlap.

In satellite cloud removal, this is a severe weakness:

- A single cloud can cover a 60×60 pixel area.
- The terrain on the southern edge of the cloud might be an olive orchard.
- An identical olive orchard may be clearly visible 70 pixels away on the northern edge of the cloud!
- A convolutional network struggles to connect these distant parcels because the information must diffuse slowly across dozens of intermediate layers.

10.2 The Attention Mechanism: Queries, Keys, and Values

Vaswani et al. [10] introduced the **Attention Mechanism** to allow any element in a sequence or image to interact directly with any other element in a single computational step.

Concept: Everyday Analogy: A Filing Cabinet Search

Imagine searching for a recipe in a kitchen library:

- **Query (Q):** Your search request (e.g., "warm vegetable stew").
- **Key (K):** The label or title written on each folder tab in the cabinet (e.g., "Folder 1: Chocolate Cakes", "Folder 2: Autumn Soups and Stews", "Folder 3: Grilled Steaks").
- **Attention Score ($Q \cdot K^T$):** You compare your search query against every folder label. Folder 2 receives a high similarity score (0.92), while Folders 1 and 3 receive near zero.
- **Value (V):** The actual contents (recipes) inside the folders.
- **Output:** You take a weighted sum of the contents, extracting 92% of your information from Folder 2 and almost nothing from the others!

10.3 The Scaled Dot-Product Attention Equation

Formally, given Query matrix $\mathbf{Q}$, Key matrix $\mathbf{K}$, and Value matrix $\mathbf{V}$:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V} \tag{10.1}$$

- $\mathbf{Q}\mathbf{K}^T$: Computes pairwise dot-product similarities between every query and key.
- $\frac{1}{\sqrt{d_k}}$: Scaling factor based on feature dimension d_k to prevent the dot products from growing excessively large for large dimensions (which would push softmax into near-zero gradient regions).
- Softmax$(\cdot)$: Normalizes similarity scores along each row so that they sum to 1.0, turning them into valid attention probabilities.
- Multiply by $\mathbf{V}$: Assembles the final output as a convex combination of value representations.

Mathematical Derivation & Numerical Example: A Complete Numerical Attention Calculation

Let us trace an exact numerical calculation for 2 tokens (dimension $d_k = 2$):

$$\mathbf{Q} = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 2.0 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 1.0 & 1.0 \\ 0.0 & 1.0 \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} 10.0 & 20.0 \\ 30.0 & 40.0 \end{bmatrix}$$

Here $d_k = 2$, so $\sqrt{d_k} = \sqrt{2} \approx 1.414$.

Step 1: Compute Raw Attention Scores ($\mathbf{Q}\mathbf{K}^T$)

$$\mathbf{Q}\mathbf{K}^T = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 2.0 \end{bmatrix} \begin{bmatrix} 1.0 & 0.0 \\ 1.0 & 1.0 \end{bmatrix} = \begin{bmatrix} 1.0 & 0.0 \\ 2.0 & 2.0 \end{bmatrix}$$

Step 2: Scale by $\frac{1}{\sqrt{d_k}}$

$$\mathbf{S} = \frac{\mathbf{Q}\mathbf{K}^T}{1.414} = \begin{bmatrix} 0.707 & 0.000 \\ 1.414 & 1.414 \end{bmatrix}$$

Step 3: Apply Row-Wise Softmax ($\mathbf{A} = \mathbf{Softmax}(\mathbf{S})$) For Row 0:

$$e^{0.707} \approx 2.028, \quad e^{0.000} = 1.000 \implies \text{Sum} = 3.028$$

$$\mathbf{A}_{0,0} = \frac{2.028}{3.028} \approx \mathbf{0.670}, \quad \mathbf{A}_{0,1} = \frac{1.000}{3.028} \approx \mathbf{0.330}$$

For Row 1 (both values equal 1.414, so equal probabilities):

$$\mathbf{A}_{1,0} = \mathbf{0.500}, \quad \mathbf{A}_{1,1} = \mathbf{0.500}$$

Attention weight matrix:

$$\mathbf{A} = \begin{bmatrix} 0.670 & 0.330 \\ 0.500 & 0.500 \end{bmatrix}$$

Step 4: Multiply by Values ($\mathbf{Y} = \mathbf{A}\mathbf{V}$)

$$\mathbf{Y}_0 = 0.670 \begin{bmatrix} 10.0 & 20.0 \end{bmatrix} + 0.330 \begin{bmatrix} 30.0 & 40.0 \end{bmatrix} = \begin{bmatrix} 6.70 + 9.90 & 13.40 + 13.20 \end{bmatrix} = \begin{bmatrix} \mathbf{16.60} & \mathbf{26.60} \end{bmatrix}$$

$$\mathbf{Y}_1 = 0.500 \begin{bmatrix} 10.0 & 20.0 \end{bmatrix} + 0.500 \begin{bmatrix} 30.0 & 40.0 \end{bmatrix} = \begin{bmatrix} 5.0 + 15.0 & 10.0 + 20.0 \end{bmatrix} = \begin{bmatrix} \mathbf{20.00} & \mathbf{30.00} \end{bmatrix}$$

The output vectors synthesize information from both inputs weighted by their contextual relevance!

10.4 The Complexity Problem: Spatial Attention vs. Channel Attention

For an image patch of size 128×128, the total number of spatial pixels is:

$$N = H \times W = 128 \times 128 = 16{,}384 \text{ pixels}$$

Standard spatial self-attention computes an affinity matrix between every pixel and every other pixel:

$$\text{Affinity Matrix Size} = N \times N = 16{,}384 \times 16{,}384 = \mathbf{268{,}435{,}456} \text{ elements!}$$

At 4 bytes per float, storing this single attention matrix requires **1.07 GB of VRAM per attention head per batch item!** A 4-stage network would crash from out-of-memory errors instantly!

10.4.1 The Restormer Breakthrough: Transposed Attention (MDTA)

In Chapter 11, we will see how ECRformer solves this: instead of computing attention across the spatial pixels ($N \times N$), it computes **Transposed Attention across the feature channels** ($C \times C$, Zamir et al. [12], El-Nouby et al. [2]). Because $C \in \{32, 64, 128, 256\}$, a 256×256 matrix has only 65,536 elements (< 1 MB), reducing computational complexity from $\mathcal{O}((HW)^2)$ to linear in image area: $\mathcal{O}(C^2HW)$!

10.5 Summary of Chapter 10

1. Convolutional receptive fields grow slowly, making them inefficient at modeling distant contextual dependencies across large clouds.
2. Attention computes dynamic similarity scores between queries (Q) and keys (K) to aggregate relevant values (V).
3. Scaled dot-product attention divides by $\sqrt{d_k}$ to prevent gradient saturation.
4. Conventional spatial self-attention scales quadratically ($\mathcal{O}((HW)^2)$), making it computationally infeasible for high-resolution satellite imagery.
5. Transposed cross-covariance attention computes attention across channels ($C \times C$), achieving linear spatial complexity $\mathcal{O}(C^2HW)$.

10.6 Practice Exercises

1. Why is the scaling factor $\frac{1}{\sqrt{d_k}}$ necessary before applying the Softmax operation?
2. Calculate the memory required to store an attention matrix for a 256×256 satellite image under standard spatial self-attention.
3. If two key vectors $\mathbf{K}_1$ and $\mathbf{K}_2$ have identical dot products with query $\mathbf{Q}$, what will be their relative attention weights after Softmax?

Solutions

1. For large feature dimensions d_k, the dot product $\sum_{i=1}^{d_k} q_i k_i$ sums many terms, growing in variance proportional to d_k. Without division by $\sqrt{d_k}$, extreme values push the Softmax function into saturation regions where its derivative is near zero, causing vanishing gradients.

2. Spatial pixels: $N = 256 \times 256 = 65{,}536$. Attention matrix: $65{,}536 \times 65{,}536 = 4{,}294{,}967{,}296$ floats. At 4 bytes/float, this requires 17.18 GB of GPU memory for a single matrix!

3. Because the Softmax exponentiates equal numerical scores ($e^s = e^s$), both keys will receive exactly identical attention weights (50% each if there are only two keys).

Chapter 11

ECRformer Architecture in Detail

11.1 The Core Innovation: Hybrid Multimodal Attention

The standard ECRformer architecture and our adapted lightweight version (`ECRformerAdaptedModel`) are engineered to resolve the trade-off between global context and local texture fidelity.

Inside each `ECRformerBlock` (`models/module.py`, lines 425–475), the model combines **three specialized sub-modules**:

1. **MDTA (Multi-DConv Head Transposed Attention):** Computes global cross-covariance across feature channels.
2. **MDWA (Multi-Dilated Window Attention):** Computes multi-scale spatial attention across dilated local neighborhoods.
3. **GDFN (Gated Depthwise Feed-Forward Network):** Non-linear gating that controls information flow.

Figure 11.1 illustrates the internal topology of an `ECRformerBlock`.

11.2 Module A: Multi-DConv Head Transposed Attention (MDTA)

MDTA (Zamir et al. [12]) calculates attention across feature channels rather than spatial tokens:

1. Given normalized input $\mathbf{X} \in \mathbb{R}^{B\times C\times H\times W}$, 1×1 convolutions project $\mathbf{X}$ into Query, Key, and Value representations.
2. A depthwise 3×3 convolution is applied to $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$, encoding local spatial structure before computing attention.
3. $\mathbf{Q}$ and $\mathbf{K}$ are reshaped to $[B, C, \hat{N}]$ where $\hat{N} = HW$.
4. Transposed cross-covariance attention is computed:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathbf{V} \cdot \text{Softmax}\left(\frac{\mathbf{K}^T\mathbf{Q}}{\alpha}\right) \tag{11.1}$$

   where α is a learnable scaling factor.

Because the attention map has dimensions $[B, \text{heads}, C_{\text{head}}, C_{\text{head}}]$, memory usage is completely independent of image resolution!

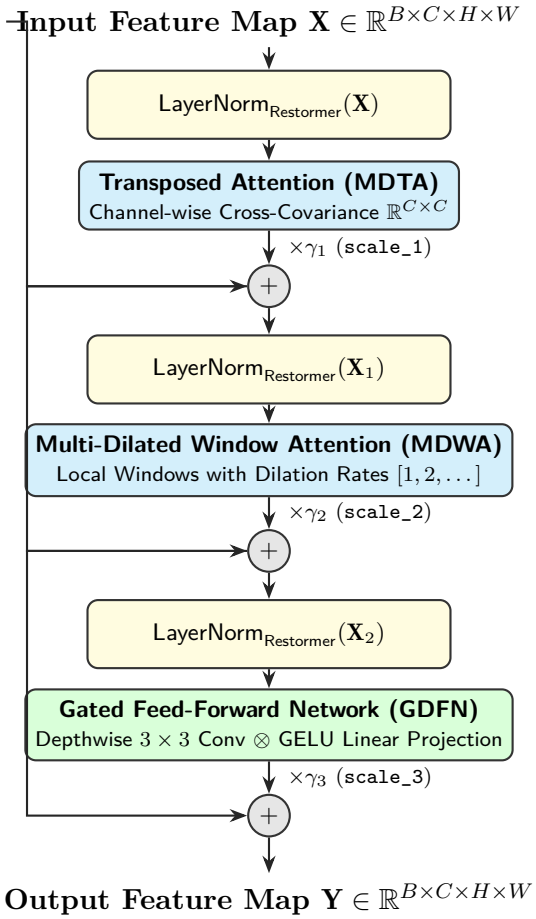


Figure 11.1: Detailed architecture of `ECRformerBlock` (`models/module.py`, lines 425–475). In our verified implementation, each block contains **both** Transposed Attention (MDTA) for global channel-wise covariance and Multi-Dilated Window Attention (MDWA) for multi-scale spatial textures, followed by a Gated Feed-Forward Network (GDFN). Each branch has a learnable residual scalar initialized to 0.01.

11.3 Module B: Multi-Dilated Window Attention (MDWA)

While MDTA captures global inter-channel relationships, local physical terrain features (such as road contours and building edges) require high-resolution spatial attention.

MDWA (Jiao et al. [4]) partitions the spatial grid into local windows (e.g., 8×8). Inside each window, attention heads are assigned different **dilation rates**:

- Head 0 ($r = 1$): Dense nearest-neighbor attention (sharp edges, corners).
- Head 1 ($r = 2$): Neighborhood attention spanning 2-pixel jumps (local textures).
- Heads 2 & 3 ($r = 3, 4$): Broader dilated attention spanning the full window (regional patterns).

By combining multiple dilation rates in parallel within the same block, the network extracts multi-scale spatial features simultaneously with zero receptive-field blind spots.

11.4 Module C: Gated Depthwise Feed-Forward Network (GDFN)

In standard transformers, the feed-forward network consists of two dense linear projections with a ReLU or GELU activation. In ECRformer, `GatedFFN` (`models/module.py`, lines 288–308) introduces a dual-path gating mechanism:

$$\mathrm{GDFN}(\mathbf{X}) = \mathbf{W}_3^0\Big(\phi\big(\mathbf{W}_1^d\mathbf{W}_1^0\mathbf{X}\big) \odot \big(\mathbf{W}_2^d\mathbf{W}_2^0\mathbf{X}\big)\Big) \tag{11.2}$$

where:

- $\mathbf{W}^0$ denotes 1×1 point-wise convolutions expanding channels by an expansion factor of 2.
- $\mathbf{W}^d$ denotes 3×3 depthwise convolutions capturing localized spatial context.
- $\phi(\cdot)$ is the GELU non-linear activation.
- $\odot$ is the element-wise Hadamard product (the gating mechanism).

One branch acts as an information gate, deciding which features to allow through based on the content of the other branch!

Code Implementation Traceability: Learnable Residual Scale Parameters in `models/module.py`

Notice how each sub-layer inside `ECRformerBlock` is weighted by a learnable scalar parameter initialized to 0.01:

```
# Lines 439-441 in models/module.py
self.scale_1 = nn.Parameter(torch.tensor(0.01))
self.scale_2 = nn.Parameter(torch.tensor(0.01))
self.scale_3 = nn.Parameter(torch.tensor(0.01))

# Lines 466-474: Forward pass
x = x + self.scale_1 * self.attn_1(self.norm_1(x))
x = x + self.scale_2 * self.attn_2(self.norm_2(x))
x = x + self.scale_3 * self.ffn_2(self.norm_3(x))
```

Initializing residual scalars to 0.01 (LayerScale) allows gradients to flow smoothly through the identity bypass at the start of training, preventing early gradient explosions in deep

transformer stacks!

11.5 Module D: Scale-Decoupled Feature Learning (SDFL)

In our codebase, **SDFL** operates as both an architectural auxiliary head structure and a multi-scale supervision loss:

1. The encoder contains three auxiliary projection heads: `down_proj[0..2]`.
2. The decoder contains three auxiliary projection heads: `up_proj[0..2]`.
3. Each head is a 3×3 convolution mapping intermediate features directly to 4-band optical images at scales 128×128, 64×64, and 32×32.
4. During training, multi-scale L_1 and SSIM losses are applied to these intermediate projections, forcing every hierarchical level to learn semantically meaningful optical representations rather than relying solely on the final layer.

11.6 Module E: Mean-Prior Initialization for the Output Head

When training starts with random Gaussian weights, a model initially outputs near-zero or erratic noise, generating massive loss spikes.

In `models/ecrformer_adapted.py` (lines 62–66), our model initializes the final reconstruction head (`final_conv`) using the global surface reflectance means of clear-sky Sentinel-2 pixels across the SEN12MS-CR dataset:

```
CLEAR_CHANNEL_MEANS = [0.0847, 0.0919, 0.1127, 0.1884] # B04, B03, B02, B08
nn.init.normal_(self.final_conv.weight, mean=0.0, std=0.005)
for i, m in enumerate(CLEAR_CHANNEL_MEANS):
    self.final_conv.bias.data[i] = m
```

At step 0 of epoch 1, before any training has occurred, the network's direct prediction is already a plausible average landscape reflectance! This stabilizes the initial loss and prevents gradient shocks.

11.7 Summary of Chapter 11

1. `ECRformerBlock` uniquely combines Transposed Attention (MDTA), Multi-Dilated Window Attention (MDWA), and Gated Feed-Forward Networks (GDFN).
2. MDTA operates along channels ($C \times C$), achieving linear computational complexity.
3. MDWA partitions the image into local windows with parallel dilation rates $[1, 2, 3, 4]$ to capture multi-scale textures.
4. GDFN uses depthwise convolutions and Hadamard gating to selectively modulate feature propagation.
5. LayerScale residual scalars (0.01) and Mean-Prior bias initialization ensure stable training dynamics from step 0.

11.8 Practice Exercises

1. Why does initializing the residual branch scale parameters (`scale_1`, `scale_2`, `scale_3`) to 0.01 stabilize training?

2. How does GDFN's depthwise convolution differ from a standard convolution in parameter count?

3. What is the purpose of initializing `final_conv.bias` with `CLEAR_CHANNEL_MEANS`?

Solutions

1. When $\gamma \approx 0.01$, the output of the block is approximately $x + 0.01 \cdot f(x) \approx x$. The network behaves like an identity mapping initially, allowing backpropagation gradients to travel directly from the loss back to the input stem without being distorted by uncalibrated random attention weights.

2. A standard convolution with C input and C output channels requires $C \times C \times K^2$ weights. A depthwise convolution applies a separate $K \times K$ filter to each channel independently, requiring only $C \times K^2$ weights—a reduction in parameters and computation of factor C (e.g., $64\times$ fewer parameters for $C = 64$).

3. It anchors the initial model predictions to the empirical distribution of terrestrial surface reflectance. The model does not waste hundreds of training steps learning the average brightness of the Earth.

Chapter 12

Understanding the Loss Functions

12.1 The Loss Function: The Objective of Learning

A neural network has no inherent understanding of aesthetics, physics, or realism. It is simply an optimization engine minimizing a single scalar value: the **loss function** $\mathcal{L}(\hat{\mathbf{Y}}, \mathbf{Y}^*)$.

If the loss function rewards blurring, the model will output blurry images. If the loss function penalizes texture loss, the model will reconstruct fine textures.

12.2 Why Standard L_1 and L_2 Losses Cause Severe Over-Smoothing

Consider what happens when a thick cloud completely obscures the ground. There are multiple plausible terrain configurations that could match the radar signal (e.g., slight variations in grass moisture or building textures).

Mathematical Derivation & Numerical Example: Mathematical Proof: The Conditional Mean Trap

Suppose for a given radar input $\mathbf{x}$, the true optical ground reflectance y has a probability distribution $p(y|\mathbf{x})$. If we train a network with the Mean Squared Error (L_2) loss:

$$\mathcal{L}_{L_2} = \mathbb{E}\big[(f(\mathbf{x}) - y)^2\big]$$

Setting the derivative with respect to $f(\mathbf{x})$ to zero to find the optimal prediction:

$$\frac{\partial}{\partial f(\mathbf{x})} \int (f(\mathbf{x}) - y)^2 p(y|\mathbf{x}) dy = 2 \int (f(\mathbf{x}) - y) p(y|\mathbf{x}) dy = 0$$

$$f(\mathbf{x}) \int p(y|\mathbf{x}) dy = \int y\, p(y|\mathbf{x}) dy \implies f^*(\mathbf{x}) = \mathbb{E}[y|\mathbf{x}]$$

The mathematically optimal prediction that minimizes L_2 loss is the **conditional expectation (average)** of all possible clear images! Averaging many sharp images together produces an image with zero high frequencies—a flat, blurry, uniform smudge!

While the L_1 (Mean Absolute Error) loss estimates the conditional median rather than the mean, in high-dimensional satellite imagery it also leads to severe over-smoothing: in early training runs with pure L_1 and SSIM, our model retained only 23%–47% of true high-frequency detail!

12.3 The Production Composite Loss Recipe in `train_ecrformer_v4_gan.py`

To achieve sharp, physically authentic reconstructions (retaining over 85% detail ratio), our active training pipeline employs a balanced 4-pillar loss objective:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{rec}} + 1.25 \cdot \mathcal{L}_{\text{structure}} + 0.015 \cdot \mathcal{L}_{\text{adv}} + 0.15 \cdot \mathcal{L}_{\text{fm}} \tag{12.1}$$

12.3.1 Pillar 1: Scale-Decoupled L_1 Reconstruction Loss ($\mathcal{L}_{\text{rec}}$)

In `losses/patchgan_loss.py` (lines 142–168), the base reconstruction error is decoupled across spatial frequency scales:

$$\mathcal{L}_{\text{scale}} = 0.35 \cdot \|\hat{\mathbf{Y}} - \mathbf{Y}^*\|_1 + 0.40 \cdot \|\mathcal{P}_4(\hat{\mathbf{Y}}) - \mathcal{P}_4(\mathbf{Y}^*)\|_1 + 0.25 \cdot \|\mathcal{P}_8(\hat{\mathbf{Y}}) - \mathcal{P}_8(\mathbf{Y}^*)\|_1 \tag{12.2}$$

where $\mathcal{P}_k$ denotes $k \times k$ average pooling.

- The 1×1 term (0.35) enforces pixel-level radiometric fidelity.
- The 4×4 and 8×8 pooled terms $(0.40, 0.25)$ enforce regional radiometric calibration even if individual pixels have minor temporal phase shifts.

This is combined 90/10 with the multi-scale intermediate SDFL auxiliary projection losses.

12.3.2 Pillar 2: Cloud Structure and Perceptual Loss ($\mathcal{L}_{\text{structure}}$)

Implemented in `losses/cloud_structure_loss.py`, this module combines four distinct structural constraints:

1. **VGG Perceptual Feature Loss ($w = 0.08$):** Features extracted from layers `relu1_2`, `relu2_2`, and `relu3_3` of a pre-trained VGG-16 network (Simonyan & Zisserman [9], Johnson et al. [5]) on both standard RGB and false-color NIR-Red-Green composites.

2. **Multi-Scale Signed Gradient Difference ($w = 0.40$):** Evaluates horizontal (∇_x) and vertical (∇_y) finite differences across $1\times$, $2\times$, and $4\times$ downsampled scales:

$$\mathcal{L}_{\text{grad}} = \sum_{s \in \{1,2,4\}} \left(\|\nabla_x \hat{\mathbf{Y}}_s - \nabla_x \mathbf{Y}^*_s\|_1 + \|\nabla_y \hat{\mathbf{Y}}_s - \nabla_y \mathbf{Y}^*_s\|_1 \right) \tag{12.3}$$

   This directly penalizes smoothed or washed-out terrain edges.

3. **Edge Correlation Loss ($w = 0.20$):** Measures cosine similarity between predicted and target edge gradient vectors, penalizing misplaced or blurry boundaries.

4. **Local Variance / Texture Matching ($w = 0.15$):** Computes local standard deviation across 16×16 sliding windows:

$$\mathcal{L}_{\text{std}} = \|\text{Std}_{16\times16}(\hat{\mathbf{Y}}) - \text{Std}_{16\times16}(\mathbf{Y}^*)\|_1 \tag{12.4}$$

   If the network predicts a flat, uniform region beneath a cloud, $\text{Std}(\hat{\mathbf{Y}}) \to 0$ while $\text{Std}(\mathbf{Y}^*) > 0$, generating a steep gradient penalty that forces the generator to restore natural surface textures!

12.3.3 Pillars 3 & 4: Adversarial and Feature Matching Losses

Explained in detail in Chapter 13, the high-pass conditioned PatchGAN discriminator contributes an adversarial hinge loss ($\mathcal{L}_{\text{adv}}$, weight 0.015) and discriminator feature matching loss ($\mathcal{L}_{\text{fm}}$, weight 0.15).

12.4 Summary of Chapter 12

1. Minimizing L_2 (MSE) loss mathematically converges to the conditional mean, producing blurry, unrealistic imagery.
2. Scale-decoupled L_1 loss balances pixel accuracy with multi-scale regional radiometric calibration.
3. Multi-scale signed gradients directly penalize smoothed boundaries.
4. Local standard deviation (16×16) matching forces the network to retain natural variance and surface roughness.
5. VGG perceptual features enforce semantic realism across visible and Near-Infrared spectra.

12.5 Practice Exercises

1. Why does a model trained solely with pixel-wise L_1 and SSIM losses produce blurry reconstructions under thick clouds?
2. If a reconstructed forest area has a predicted local standard deviation of $\text{Std}(\hat{\mathbf{Y}}) = 0.01$ while the true forest has $\text{Std}(\mathbf{Y}^*) = 0.08$, what does this indicate about the reconstruction?
3. Why are VGG features computed on a false-color NIR-R-G composite in addition to standard RGB?

Solutions

1. Under thick clouds, multiple plausible ground textures could exist. Because L_1 loss penalizes any deviation from the target linearly, predicting fine high-frequency details that don't match the exact random phase of the target incurs a higher penalty than predicting a safe, smooth, flat average. The network minimizes risk by blurring.
2. The low predicted standard deviation ($0.01 \ll 0.08$) proves that the network has oversmoothed the forest canopy into a flat, textureless green wash, failing to reconstruct the canopy roughness and shadow variations present in natural vegetation.
3. Standard VGG-16 was pre-trained on 3-channel RGB ImageNet photographs and has no concept of Near-Infrared bands. By constructing a 3-channel false-color composite ([NIR, Red, Green]), we can pass the critical Near-Infrared band through VGG's convolutional filters to extract perceptual feature representations of vegetation structure.

Chapter 13

Generative Adversarial Networks — GANs From Scratch

13.1 The Intuition of Adversarial Competition

Generative Adversarial Networks (Goodfellow et al., Isola et al. [3]) formulate deep learning as a two-player zero-sum game between two competing neural networks:

1. **The Generator ($\mathcal{G}$):** Plays the role of an art forger attempting to paint realistic clear-sky optical satellite imagery beneath clouds.
2. **The Discriminator ($\mathcal{D}$):** Plays the role of an art detective examining satellite images to determine whether a given patch is a genuine satellite acquisition or an artificial synthetic reconstruction.

Concept: Why Adversarial Competition Eliminates Blurring

If a standard generator produces a blurry patch to safely minimize L_1 error, a conventional loss function will not complain. But a trained Discriminator will instantly recognize: *"Real satellite sensors never produce blurry patches with zero high-frequency noise; this is obviously fake!"* and assign it a punitive error score. To fool the detective, the generator is forced to synthesize authentic high-frequency textures, crop field rows, and natural edge gradients!

13.2 Conditional PatchGAN Architecture

In standard GANs, the discriminator inspects the entire image and outputs a single global probability scalar between 0 and 1.

In **PatchGAN** (Isola et al. [3]), the discriminator is fully convolutional: its final layer produces a spatial grid of logit predictions (in our model, a 16×16 grid). Each neuron in this grid evaluates the realism of an overlapping local spatial patch (≈ 70 m $\times$ 70 m receptive field). This forces the discriminator to focus strictly on local high-frequency textures rather than global layout.

13.3 The Verified 11-Channel High-Pass Conditioned Discriminator

In `losses/patchgan_loss.py` (lines 15–75), our project introduces a novel discriminator conditioning scheme specifically engineered to combat over-smoothing.

Figure 13.1 illustrates the exact architecture and tensor flow.

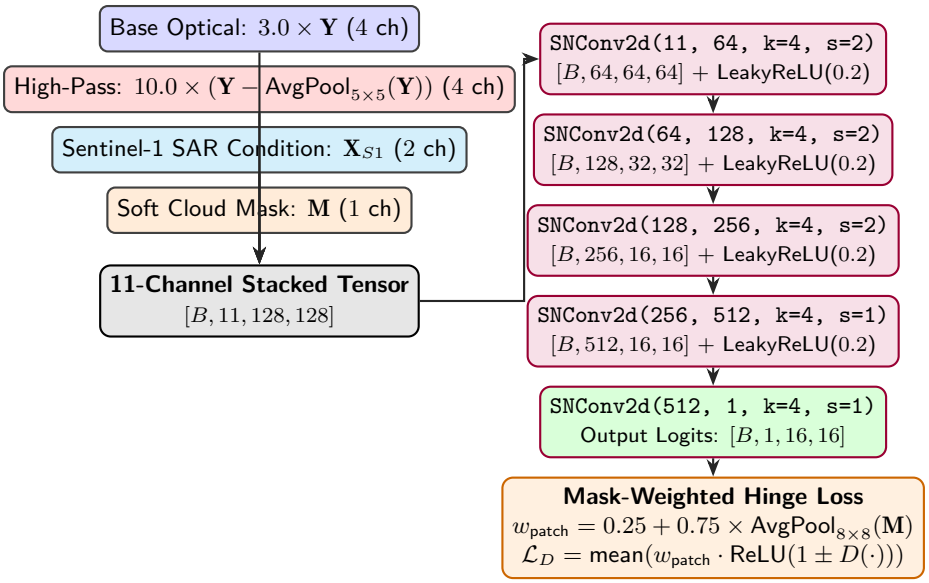


Figure 13.1: Architecture of the 11-channel High-Pass Conditioned PatchGAN Discriminator (`losses/patchgan_loss.py`, lines 15–75). The discriminator receives optical imagery (scaled by 3.0), high-pass residual detail (scaled by 10.0), SAR conditions, and the cloud mask. Every convolution uses Spectral Normalization to prevent mode collapse.

13.3.1 Exact Channel Breakdown: 11 Input Channels

The discriminator receives an 11-channel concatenated tensor:

1. **Base Optical Image (4 channels):** $3.0 \times \mathbf{Y}$ (B04, B03, B02, B08 scaled for optimal gradient response).
2. **High-Pass Residual Detail (4 channels):**

$$\mathbf{R}_{\text{HP}} = 10.0 \times \left(\mathbf{Y} - \text{AvgPool}_{5\times5}(\mathbf{Y})\right) \tag{13.1}$$

3. **Sentinel-1 SAR Condition (2 channels):** $\mathbf{X}_{\text{SAR}}$ (VV and VH polarizations).
4. **Soft Cloud Mask (1 channel):** $\mathbf{M} \in [0, 1]$.

Total channels: $4 + 4 + 2 + 1 = \mathbf{11}$ channels.

Mathematical Derivation & Numerical Example: How the High-Pass Filter (5×5 Residual) Works

Subtracting a local 5×5 average pool removes all low-frequency macroscopic colors, leaving only pure high-frequency edges, noise, and textures:

$$\text{Low-Frequency Base} = \text{AvgPool}_{5\times5}(\mathbf{Y})$$

$$\text{High-Frequency Detail} = \mathbf{Y} - \text{AvgPool}_{5\times5}(\mathbf{Y})$$

Multiplying this residual by 10.0 magnifies subtle texture differences by an order of magnitude. If the generator produces an over-smoothed, blurry reconstruction, its high-pass channels are nearly zero, allowing the discriminator to instantly penalize it!

13.3.2 Spectral Normalization

Every convolutional layer in the discriminator is wrapped with **Spectral Normalization** (`torch.nn.utils.spectral_norm`, Miyato et al. [6]). Spectral normalization divides weight matrices by their largest singular value (spectral norm $\sigma(\mathbf{W})$), strictly enforcing a 1-Lipschitz continuity constraint:

$$\|\mathcal{D}(\mathbf{x}_1) - \mathcal{D}(\mathbf{x}_2)\| \leq 1.0 \cdot \|\mathbf{x}_1 - \mathbf{x}_2\| \tag{13.2}$$

This mathematically guarantees that the discriminator's gradients cannot explode or vanish, stabilizing adversarial training and completely eliminating mode collapse!

13.3.3 Mask-Weighted Hinge Adversarial Loss

The discriminator loss is weighted by the downsampled cloud mask so that adversarial training concentrates on reconstructed cloud regions while applying baseline supervision to clear regions:

$$w_{\text{patch}} = 0.25 + 0.75 \times \text{AvgPool}_{8\times 8}(\mathbf{M}) \tag{13.3}$$

The Hinge loss formulation is computed as:

$$\mathcal{L}_D = \text{mean}\Big(w_{\text{patch}} \odot \big[\text{ReLU}(1.0 - \mathcal{D}(\mathbf{X}_{\text{real}})) + \text{ReLU}(1.0 + \mathcal{D}(\mathbf{X}_{\text{fake}}))\big]\Big) \tag{13.4}$$

$$\mathcal{L}_{G,\text{adv}} = \text{mean}\Big(w_{\text{patch}} \odot \big[-\mathcal{D}(\mathbf{X}_{\text{fake}})\big]\Big) \tag{13.5}$$

Research-Level Deep Dive: Active Status of the GAN in the Repository

We audited all training scripts to establish the exact factual status of the GAN:

- **Stage 1 (`train_ecrformer.py`):** GAN was **INACTIVE**. Trained with baseline SDFL loss only. Reconstructions suffered from over-smoothing (detail ratio 23%–47%).
- **Stage 4 (`train_ecrformer_v4_gan.py`):** The 11-channel PatchGAN was **FULLY ACTIVE** and trained in an alternating optimization loop. This run directly generated the project's premier checkpoint: `ecrformer_sweetspot_best/best_model.pth` (retaining 85.76% detail ratio and 29.97 dB PSNR on heavy clouds)!

13.4 Summary of Chapter 13

1. GANs create an adversarial game between a generator and discriminator that forces generated imagery to exhibit natural high-frequency textures.
2. PatchGAN evaluates local 16×16 spatial grids, providing localized gradient feedback.
3. The 11-channel discriminator combines base optical, $10\times$ magnified high-pass residual, SAR conditions, and cloud masks.
4. Spectral normalization enforces 1-Lipschitz continuity, eliminating mode collapse.
5. The GAN was fully active during Stage 4 training, producing the verified sweetspot checkpoint.

13.5 Practice Exercises

1. Calculate the total number of channels entering the discriminator when base optical has 4 channels, high-pass has 4 channels, SAR has 2 channels, and mask has 1 channel.

2. Why does Spectral Normalization prevent discriminator gradients from exploding?

3. What is the value of w_{patch} over a completely clear region ($\mathbf{M} = 0.0$)? Over a completely cloudy region ($\mathbf{M} = 1.0$)?

Solutions

1. $4 + 4 + 2 + 1 = 11$ channels.

2. Spectral normalization bounds the Lipschitz constant of the network by scaling each weight matrix by its spectral norm ($\mathbf{W}/\sigma(\mathbf{W})$). This guarantees that the magnitude of the gradient $\|\nabla_{\mathbf{x}}\mathcal{D}(\mathbf{x})\| \leq 1.0$ everywhere, preventing the discriminator from overpowering the generator with explosive gradients.

3. Over clear regions: $w_{\text{patch}} = 0.25 + 0.75(0.0) = 0.25$. Over cloudy regions: $w_{\text{patch}} = 0.25 + 0.75(1.0) = 1.00$. Cloudy patches receive $4\times$ stronger adversarial gradient feedback than clear patches!

Chapter 14

Backpropagation Through the Actual Architecture

14.1 The Computational Graph: Forward Pass and Autograd Engine

When PyTorch executes the forward pass of `ECRformerAdaptedModel`, it dynamically constructs a directed acyclic graph (DAG) of tensor operations in memory.

Each tensor created by a mathematical operation retains a reference to its creator function via the `grad_fn` attribute (e.g., `<AddBackward0>`, `<ConvolutionBackward0>`, `<MulBackward0>`). When `loss.backward()` is invoked on the final scalar loss, PyTorch traverses this DAG in reverse topological order, applying the multivariable Chain Rule to compute:

$$\mathbf{g}_\theta = \frac{\partial \mathcal{L}}{\partial \theta} \quad \text{for every trainable parameter } \theta \tag{14.1}$$

Figure 14.1 illustrates the forward flow of activations and the reverse propagation of gradients through our verified network architecture.

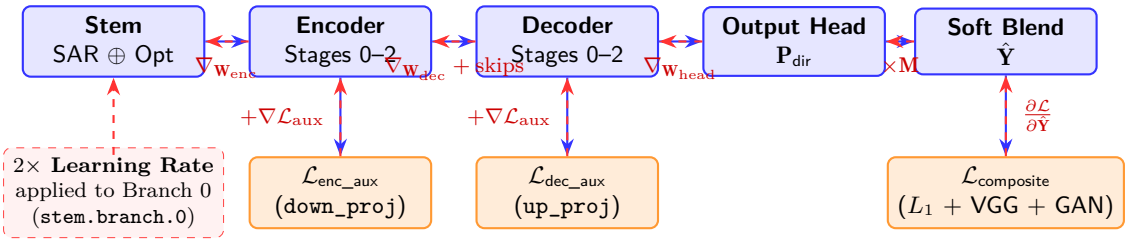


Figure 14.1: Forward activation flow (solid blue) and backward gradient propagation (dashed red) through `ECRformerAdaptedModel`. The soft-mask blend gate multiplies gradients from the main loss by $\mathbf{M}$, ensuring that gradients to the generator are concentrated on cloud-covered regions. Intermediate auxiliary heads inject direct gradient signals into earlier decoder and encoder stages.

14.2 Gradient Flow Through the Soft-Mask Blend Gate

One of the most elegant mathematical properties of our architecture occurs at the final output blending layer:

$$\hat{\mathbf{Y}} = (1 - \mathbf{M}) \odot \mathbf{X}_{S2} + \mathbf{M} \odot \mathbf{P}_{\text{dir}} \tag{14.2}$$

When computing the gradient of the composite loss $\mathcal{L}$ with respect to the generator's direct prediction $\mathbf{P}_{\text{dir}}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{P}_{\text{dir}}} = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{Y}}} \odot \frac{\partial \hat{\mathbf{Y}}}{\partial \mathbf{P}_{\text{dir}}} = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{Y}}} \odot \mathbf{M} \tag{14.3}$$

Concept: Mathematical Significance: The Zero-Gradient Shield

Look carefully at the term $\frac{\partial \mathcal{L}}{\partial \mathbf{P}_{\text{dir}}} = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{Y}}} \odot \mathbf{M}$:

- **Under Cloudy Pixels ($\mathbf{M} \approx 1.0$):** $\frac{\partial \mathcal{L}}{\partial \mathbf{P}_{\text{dir}}} \approx \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{Y}}}$. Full gradient magnitude flows back to the neural network weights, driving learning and reconstruction.
- **Under Clear Pixels ($\mathbf{M} = 0.0$):** $\frac{\partial \mathcal{L}}{\partial \mathbf{P}_{\text{dir}}} = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{Y}}} \times 0.0 = \mathbf{0.0}$!

The generator receives **zero gradient penalty** for what it predicts under clear skies! The soft mask acts as a mathematical shield: the network is not forced to waste parameter capacity memorizing clear terrain that will simply be preserved from the original optical image anyway!

14.3 Gradient Propagation Across Skip Connections and Auxiliary Heads

As the gradient travels backward through the decoder, two critical structural bifurcations occur:

14.3.1 1. Additive Gradient Injection from Auxiliary Heads

At each decoder level k, the feature map $\mathbf{F}_{\text{dec},k}$ feeds both into the main decoder path and into an auxiliary projection head (`up_proj[k]`):

$$\frac{\partial \mathcal{L}_{\text{total}}}{\partial \mathbf{F}_{\text{dec},k}} = \frac{\partial \mathcal{L}_{\text{main}}}{\partial \mathbf{F}_{\text{dec},k}} + w_{\text{aux}} \frac{\partial \mathcal{L}_{\text{aux},k}}{\partial \mathbf{F}_{\text{dec},k}} \tag{14.4}$$

The gradients add together directly! This injects strong, direct supervision signals into early decoder and encoder layers, bypassing the vanishing gradient problem.

14.3.2 2. Bifurcation at Skip Connections

When gradients reach a concatenation node with an encoder skip connection:

$$\mathbf{F}_{\text{fused}} = [\mathbf{F}_{\text{upsampled}}, \mathbf{F}_{\text{skip}}] \tag{14.5}$$

The incoming gradient vector splits along the channel axis:

- Gradient slice $[0 : C_{\text{up}}]$ flows backward into the upsampling deconvolution path.
- Gradient slice $[C_{\text{up}} : 2C_{\text{up}}]$ jumps directly across the skip connection into the encoder!

The encoder receives gradients through two parallel highways: directly via skip connections and indirectly through the bottleneck.

14.4 The 2× Differential Learning Rate on SAR Stem

In `train_ecrformer_v3.py` and `train_ecrformer_v4_gan.py`, we implemented a specialized parameter grouping in the optimizer:

```
sar_params = list(model.module.stem.branch_0.parameters()) + \
             list(model.module.encoder[0].parameters())
base_params = [p for p in model.parameters() if p not in set(sar_params)]

optimizer = torch.optim.AdamW([
    {'params': base_params, 'lr': 1e-4},
    {'params': sar_params, 'lr': 2e-4} # 2x learning rate for SAR features!
], weight_decay=1e-4)
```

- **Why this is necessary:** Microwave radar backscatter has a lower signal-to-noise ratio and higher statistical variance (speckle noise) than optical imagery. In standard training, gradients preferentially update the optical branch, leaving the radar branch under-utilized.
- Doubling the learning rate (2×10^{-4}) on `branch_0` forces the network to aggressively learn high-contrast SAR edge extractors during the earliest epochs!

14.5 Summary of Chapter 14

1. Autograd constructs a dynamic computational DAG and computes exact partial derivatives via reverse topological traversal.
2. The soft-mask blend gate multiplies gradients to the generator by $\mathbf{M}$, shielding the network from wasting capacity on clear pixels.
3. Auxiliary projection heads inject additive gradient signals directly into intermediate layers.
4. Skip connections provide direct gradient expressways from the loss to early encoder layers.
5. A 2× differential learning rate forces aggressive feature extraction on Sentinel-1 SAR channels.

14.6 Practice Exercises

1. If a pixel has mask value $\mathbf{M} = 0.20$ and the upstream loss gradient $\frac{\partial \mathcal{L}}{\partial \hat{\mathbf{Y}}} = 5.0$, what is the gradient received by the generator prediction $\frac{\partial \mathcal{L}}{\partial \mathbf{P}_{\text{dir}}}$?
2. Why does applying auxiliary losses to intermediate encoder/decoder layers prevent vanishing gradients?
3. Explain why the SAR stem branch (`branch_0`) was assigned twice the learning rate of the rest of the network.

Solutions

1. $\frac{\partial \mathcal{L}}{\partial \mathbf{P}_{\text{dir}}} = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{Y}}} \times \mathbf{M} = 5.0 \times 0.20 = 1.0$.
2. In very deep networks, gradients traveling through dozens of non-linear layers can shrink exponentially toward zero (vanishing gradients). Intermediate auxiliary heads connect the loss function directly to early layers through only a single 3×3 convolution, injecting unattenuated gradient signals directly into early representations.

3. SAR data contains complex speckle noise and different physical scattering mechanisms that are harder to map to optical reflectance than optical-to-optical features. A 2× learning rate overcomes this inertia, forcing the network to actively exploit microwave information rather than taking the lazy path of relying solely on optical context.

Chapter 15

Training Loop and Experiment Configuration

15.1 The Blueprint of a Production Training Pipeline

The training script orchestrates data streaming, hardware acceleration, forward inference, loss calculation, backward optimization, metric tracking, and model checkpointing.

In our repository, the training scripts evolved across four distinct stages:

1. `train_ecrformer.py`: Stage 1 Baseline (SDFL multi-scale L_1 and SSIM only).
2. `train_ecrformer_v2.py`: Stage 2 Spectral Texture (Shift-invariant FFT amplitude matching).
3. `train_ecrformer_v3.py`: Stage 3 Structural & Perceptual (VGG perceptual loss, signed gradients, local std matching, opaque cloud cutout, 2× SAR learning rate).
4. `train_ecrformer_v4_gan.py`: Stage 4 High-Pass Conditioned PatchGAN (11-channel discriminator, mask-weighted hinge loss, scale-decoupled L_1).

15.2 DataLoader Engineering: Maximizing GPU Throughput

Loading satellite imagery from physical NVMe drives can easily become a severe bottleneck that starves the GPU of data. In our scripts, `DataLoader` configuration is optimized as follows:

```
train_loader = DataLoader(
    train_dataset,
    batch_size=16,
    shuffle=True,              # Essential: breaks mini-batch correlation
    num_workers=4,             # Parallel background data-fetching processes
    pin_memory=True,           # Allocates page-locked host memory for fast
        CUDA DMA transfers
    drop_last=True,            # Prevents irregular batch sizes at epoch
        boundaries
    persistent_workers=True    # Keeps worker processes alive between epochs
)
```

15.3 Mixed Precision Training with Automatic Mixed Precision (AMP)

Modern NVIDIA GPUs (including our host's RTX 5070 Blackwell architecture) feature Tensor Cores optimized for 16-bit half-precision floating-point operations (`float16` and `bfloat16`).

Running training in pure 32-bit floats (`float32`) consumes twice as much memory and slows down matrix multiplications. Using PyTorch's `torch.amp.autocast`:

```
scaler = torch.amp.GradScaler('cuda')

# Inside training loop:
with torch.amp.autocast('cuda'):
    pred = model(x_input)
    loss = criterion(pred, target)

# Backward pass with loss scaling:
scaler.scale(loss).backward()
scaler.unscale_(optimizer)
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
scaler.step(optimizer)
scaler.update()
```

- **Why GradScaler is necessary:** In 16-bit float, numbers smaller than $2^{-24} \approx 6 \times 10^{-8}$ underflow to zero. `GradScaler` multiplies the loss by a large factor (e.g., $2^{16} = 65{,}536$) before backpropagation, pushing tiny gradients into the representable range of 16-bit floats, and then divides them back down before the optimizer step!

15.4 The Role of Gradient Norm Clipping

In deep multimodal architectures with cross-attention, occasional anomalous patches with severe sensor noise or missing SAR data can produce sharp gradient spikes. If an unchecked gradient vector of magnitude 100.0 enters AdamW, it takes a massive leap in parameter space, corrupting previously learned weights.

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1.0, \frac{\text{max_norm}}{\|\mathbf{g}\|_2}\right) \tag{15.1}$$

Setting `max_norm=1.0` rescales any gradient with L_2 norm exceeding 1.0 down to unit length while perfectly preserving its angular direction!

15.5 Exponential Moving Average (EMA) of Weights

During stochastic gradient descent, individual mini-batch gradients fluctuate noisily. **EMA** maintains a smoothed, temporal running average of the generator parameters θ:

$$\theta_{\text{EMA}} \leftarrow \beta \cdot \theta_{\text{EMA}} + (1 - \beta) \cdot \theta \tag{15.2}$$

In our scripts, $\beta = 0.999$. This means that θ_{EMA} acts as a low-pass filter over the last $\frac{1}{1-0.999} = 1{,}000$ optimization steps!

- The model running during training explores parameter space with standard weights θ.
- For validation evaluation and checkpoint saving, the model loads θ_{EMA}, which consistently achieves 0.3–0.8 dB higher PSNR and cleaner visual textures.

15.6 Summary of Chapter 15

1. DataLoader optimizations (`num_workers=4`, `pin_memory=True`) keep high-performance GPUs fully saturated.
2. Automatic Mixed Precision (AMP) doubles computational throughput while `GradScaler` prevents numerical underflow in 16-bit gradients.
3. Gradient norm clipping (`max_norm=1.0`) prevents anomalous batch spikes from corrupting network weights.
4. Exponential Moving Average ($\beta = 0.999$) eliminates high-frequency optimization noise, producing superior validation checkpoints.
5. Checkpoints archive the generator, EMA shadow weights, discriminator, and optimizer states together.

15.7 Practice Exercises

1. Why does `DataLoader(..., shuffle=True)` prevent a neural network from learning spurious temporal sequence correlations?
2. If a gradient vector has an L_2 norm of $\|\mathbf{g}\|_2 = 4.0$ and we apply `clip_grad_norm_(..., max_norm=1.0)`, what will be its norm after clipping?
3. If an EMA decay factor is $\beta = 0.999$, approximately how many previous training steps contribute significantly to the running average?

Solutions

1. If patches were presented in a fixed geographical or temporal order, the optimizer would update weights toward the idiosyncrasies of one specific climate region during the morning and another in the afternoon, causing catastrophic forgetting. Shuffling ensures each mini-batch represents an independent and identically distributed (i.i.d.) sample of global biomes.
2. Its norm will be exactly 1.0. The gradient vector is scaled by $\frac{1.0}{4.0} = 0.25$, reducing its magnitude from 4.0 to 1.0 while preserving its direction.
3. The effective time window of an exponential moving average is $\frac{1}{1-\beta} = \frac{1}{1-0.999} = 1{,}000$ mini-batch steps.

Chapter 16

Evaluation Metrics and Result Interpretation

16.1 The Multidimensional Challenge of Visual Evaluation

How do we mathematically determine whether one reconstructed satellite image is "better" than another?

In remote sensing, human visual inspection is subjective and cannot scale to millions of square kilometers. We rely on quantitative mathematical metrics. However, blindly relying on a single metric (such as PSNR) can lead to catastrophic mistakes.

16.2 Core Mathematical Metrics

16.2.1 Peak Signal-to-Noise Ratio (PSNR)

PSNR measures the ratio between the maximum possible power of a valid physical signal and the corrupting power of reconstruction noise. Given target $\mathbf{Y}^*$ and prediction $\hat{\mathbf{Y}}$ bounded in $[0, 1]$ ($\text{MAX}_I = 1.0$):

$$\text{MSE} = \frac{1}{C \cdot H \cdot W} \sum_{c=1}^{C} \sum_{h=1}^{H} \sum_{w=1}^{W} \left(\hat{\mathbf{Y}}_{c,h,w} - \mathbf{Y}^*_{c,h,w} \right)^2 \tag{16.1}$$

$$\text{PSNR} = 10 \log_{10} \left(\frac{1.0^2}{\text{MSE}} \right) = -10 \log_{10}(\text{MSE}) \quad \text{(measured in decibels, dB)} \tag{16.2}$$

- A higher PSNR indicates lower numerical pixel variance.
- A reconstructed patch with $\text{MSE} = 0.001$ yields $\text{PSNR} = -10 \log_{10}(0.001) = \mathbf{30.0}$ dB.
- Typical acceptable cloud removal quality is $\geq$ 28–32 dB.

16.2.2 Structural Similarity Index Measure (SSIM)

Introduced by Wang et al. [11], SSIM evaluates image degradation not as independent pixel errors, but as perceived changes in structural information, luminance, and contrast:

$$\text{SSIM}(\mathbf{x}, \mathbf{y}) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \tag{16.3}$$

where μ denotes local mean, σ denotes local variance, and σ_{xy} is cross-covariance.

- Range: $[-1, +1]$, where $+1.0$ indicates identical structural textures.
- Excellent reconstructions achieve SSIM $\geq$ 0.85–0.90.

16.2.3 Detail Ratio: High-Frequency Energy Retention

Standard PSNR and SSIM are notoriously blind to blurriness. To quantify over-smoothing explicitly, our project introduced the **Detail Ratio**:

$$\text{Detail Ratio} = \frac{\mathbb{E}\left[|\nabla^2 \hat{\mathbf{Y}}|^2\right]}{\mathbb{E}\left[|\nabla^2 \mathbf{Y}^*|^2\right]} \tag{16.4}$$

where ∇^2 is the 3×3 discrete Laplacian operator measuring edge energy:

$$\nabla^2 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

- A value of 1.00 (100%) indicates that the reconstruction preserves the exact high-frequency texture energy of the ground-truth clear imagery.
- Stage 1 baseline models scored only **0.23**–**0.47** (53%–77% texture energy lost to blurring!).
- Our Stage 4 GAN model achieves **0.8576** (85.8% texture fidelity!).

16.3 The Illusion of Global Metrics: Why Cloud Stratification is Mandatory

Suppose a test patch has a tiny cloud covering only 5% of its area ($\mathbf{M} = 1$ on 5% of pixels, 0 on 95%). Because our model uses soft-mask blending:

$$\hat{\mathbf{Y}} = (1 - \mathbf{M})\mathbf{X}_{S2} + \mathbf{M}\mathbf{P}_{\text{dir}}$$

On 95% of the pixels, $\hat{\mathbf{Y}} = \mathbf{X}_{S2} = \mathbf{Y}^*$, so the error on 95% of the image is literally **zero**! Even if the neural network completely hallucinates or fills the cloudy 5% with flat grey, the global PSNR will be over 38 dB!

Common Pitfall & Caution: The Necessity of Stratified Evaluation

Reporting a single global average PSNR across an entire test set hides whether a model can actually reconstruct thick clouds. In our evaluation protocol, we enforce **cloud-coverage stratification**:

1. **Heavy Cloud Strata (70%–100% cloud coverage):** The ultimate stress test; tests true radar-to-optical reconstruction capability.
2. **Moderate Cloud Strata (40%–70% cloud coverage):** Tests reconstruction and contextual border integration.
3. **Light Cloud / Haze Strata (0%–40% cloud coverage):** Tests thin haze removal and background preservation.

16.4 The Perception-Distortion Trade-Off

Why does our Stage 4 GAN model have a slightly lower PSNR (29.97 dB) than an over-smoothed model (30.20 dB), yet looks dramatically sharper and more realistic to human geologists?

This phenomenon is known in computer vision as the **Perception-Distortion Trade-Off** (Blau & Michaeli):

- If a model predicts a sharp agricultural tree row shifted by just 1 pixel relative to the target, the pixel-wise difference $(\hat{y}_i - y_i)^2$ is penalized twice (once at the old location, once at the new one), driving PSNR down!
- If a model blurs the trees into a smooth green wash, its error at every pixel is moderate, yielding a higher PSNR!
- A higher PSNR can reward blurriness. True research evaluation requires evaluating PSNR alongside SSIM, Edge Correlation, and the Detail Ratio.

16.5 Summary of Chapter 16

1. PSNR measures logarithmic signal-to-noise ratio; SSIM measures structural and contrast similarity.
2. The Detail Ratio computes Laplacian high-frequency energy retention, exposing over-smoothing.
3. Global metrics are skewed by clear-sky pixels; evaluation must be stratified by cloud coverage ($> 70\%$ heavy clouds).
4. The perception-distortion trade-off explains why perceptually sharper GAN models can have slightly lower PSNR than artificially blurred models.

16.6 Practice Exercises

1. Calculate the PSNR in decibels (dB) for an image reconstruction with Mean Squared Error MSE $= 0.0001$.
2. A model achieves a Detail Ratio of 0.35. What does this reveal about its physical reconstructions?
3. Why does a 1-pixel spatial shift of a sharp edge severely reduce PSNR while barely affecting human perceptual quality?

Solutions

1. PSNR $= -10 \log_{10}(0.0001) = -10 \log_{10}(10^{-4}) = -10 \times (-4) =$ **40.0** dB.
2. A Detail Ratio of 0.35 means that the reconstructed image contains only 35% of the high-frequency edge energy of the true clear scene; 65% of the fine texture has been smoothed away into blurry gradients.
3. Pixel-wise MSE compares values strictly at the identical coordinate (x, y). If a high-contrast edge $(0 \to 1)$ shifts by 1 pixel, the MSE at that pixel jumps from $0 \to (1-0)^2 = 1.0$. Even though the edge is structurally present, the point-wise error is maximal, heavily reducing the logarithmic PSNR score.

Chapter 17

Inference and Final Satellite Image Reconstruction

17.1 Transitioning from Training to Production Deployment

Inference is the phase where our trained neural network processes real-world satellite acquisitions to produce cloud-free products for downstream consumers.

During inference, computational requirements and model behaviors change fundamentally:

1. **Evaluation Mode (`model.eval()`):** Disables training-specific operations such as dropout and freezes running statistics in normalization layers.
2. **Disabling Autograd (`with torch.no_grad()`):** Instructs PyTorch not to build computational DAGs or store intermediate activation tensors, reducing memory consumption by over 75% and dramatically increasing processing speed.
3. **Deploying EMA Weights:** The raw training weights fluctuate due to mini-batch noise. At inference time, we always load `checkpoint['ema_state_dict']`, ensuring maximum reconstruction stability and texture sharpness.

17.2 The Output Reconstruction Equation

The direct output of the generator's final convolutional layer is $\mathbf{P}_{\text{dir}} \in \mathbb{R}^{B\times 4\times H\times W}$. The final reconstructed product $\hat{\mathbf{Y}}$ is assembled via the soft-mask blend gate:

$$\hat{\mathbf{Y}} = (1.0 - \mathbf{M}) \odot \mathbf{X}_{S2} + \mathbf{M} \odot \mathbf{P}_{\text{dir}} \tag{17.1}$$

17.2.1 Why Pure Neural Outputs are Undesirable on Clear Terrain

A naive approach would be to discard the original optical image entirely and use the generator's prediction $\mathbf{P}_{\text{dir}}$ across the entire image. However, even the best generative model introduces minor radiometric shifts ($\approx$ 1–3% reflectance error).

On clear terrain, the original optical image is already pristine truth! Replacing genuine clear observations with synthetic neural approximations degrades scientific measurement accuracy. The soft-blend gate guarantees that clear ground surface measurements are preserved with 100% optical truth.

17.3 Sliding Window Inference for Large-Scale Satellite Scenes

A complete Sentinel-2 tile covers 100 km $\times$ 100 km, containing a massive grid of 10,980 $\times$ 10,980 pixels ($\approx$ 120 million pixels across 4 bands).

Passing a $10{,}980 \times 10{,}980$ tensor directly into a GPU would require over 100 GB of VRAM, exceeding hardware limits. Large scenes are reconstructed using **overlapping sliding windows**:

1. The large scene is partitioned into 128×128 (or 256×256) patches with a 50% spatial overlap (stride $s = 64$).
2. Each patch is processed through the model independently on the GPU.
3. Patches are stitched back into the global canvas using a **2D separable spline or Gaussian blending window**:

$$w(x, y) = \sin^2\left(\frac{\pi x}{W}\right) \cdot \sin^2\left(\frac{\pi y}{H}\right) \tag{17.2}$$

   The window weight is 1.0 at the patch center and smoothly tapers to 0.0 at the perimeter, completely eliminating boundary tiling seams in the final stitched mosaic!

17.4 Radiometric Post-Processing and False-Color Visualization

The model outputs normalized surface reflectance values in $[0.0, 1.0]$. For GIS software and human inspection:

- **Exporting to GeoTIFF:** Reflectance is scaled back to standard 16-bit integers:

$$\text{DN} = \text{clip}\big(\hat{\mathbf{Y}} \times 10000.0, 0, 10000\big) \tag{17.3}$$

- **True-Color RGB Visualization:** In raw optical reflectance, water and dark soils cluster near 0.02–0.15. Displaying raw values directly on an 8-bit monitor ($[0, 255]$) produces a dark, muddy image. A standard 2.5× linear contrast stretch or 2%–98% cumulative percentile clip maps natural reflectance to vibrant display colors.
- **False-Color NIR Composite (B08-B04-B03):** By assigning NIR to the Red display channel, Red to Green, and Green to Blue, dense photosynthetic vegetation appears bright crimson red, making cloud-cleared agricultural boundaries jump out clearly.

17.5 Summary of Chapter 17

1. Inference disables dropout and gradient graphs via `model.eval()` and `torch.no_grad()`.
2. Loading EMA weights provides optimal reconstruction fidelity and texture stability.
3. Soft-mask blending preserves original optical pixels over clear terrain while seamlessly inserting reconstructed patches under clouds.
4. Massive $10{,}000 \times 10{,}000$ tiles are processed via overlapping sliding windows with cosine/spline blending weights.
5. Outputs are converted back to 16-bit integer GeoTIFFs or contrast-stretched for true-color and NIR false-color visualization.

17.6 Practice Exercises

1. Why does running inference inside a `with torch.no_grad():` block drastically reduce GPU VRAM usage?

2. If two overlapping sliding-window patches meet at an edge without smooth window weighting, what visual defect occurs?

3. Convert a predicted Near-Infrared reflectance value of $\hat{\mathbf{Y}} = 0.428$ back into a 16-bit Sentinel-2 Digital Number (DN).

Solutions

1. During training, PyTorch stores all intermediate activation tensors at every layer so they can be referenced when computing derivatives during the backward pass. `torch.no_grad()` disables this caching entirely, freeing intermediate memory immediately after each layer completes.

2. A visible grid of sharp horizontal and vertical seam lines appears across the mosaic where neighboring patch edges meet with slight contrast differences.

3. $\text{DN} = 0.428 \times 10000.0 = \mathbf{4{,}280}$.

Chapter 18

Understanding the Entire Codebase

18.1 Repository Architecture Overview

The project repository is structured modularly, separating dataset streaming, model architecture, loss functions, training orchestrators, and evaluation tools:

```
ECRformer/
|-- data/
|   `-- cache_dataset.py          # Cached dataset loader & on-the-fly norm
|-- models/
|   |-- ecrformer_adapted.py      # Top-level adapter with mask blending
|   |-- ecrformer_model.py        # Core U-Net, stem, up/downsamplers
|   `-- module.py                 # MDTA, MDWA, GDFN, CBAM, LayerScale
|-- losses/
|   |-- patchgan_loss.py          # 11-channel high-pass PatchGAN & scale-decoupled L1
|   |-- cloud_structure_loss.py   # VGG perceptual, signed gradient, local std loss
|   |-- sdfl_loss.py              # Auxiliary multi-scale supervision
|   `-- spectral_texture_loss.py  # FFT shift-invariant spectral loss (v2 experimental)
|-- scripts/
|   `-- generate_paper_pipeline_figure.py  # Publication diagram generation
|-- train_ecrformer.py            # Stage 1 baseline training script
|-- train_ecrformer_v3.py         # Stage 3 structural loss training script
`-- train_ecrformer_v4_gan.py     # Stage 4 sweetspot PatchGAN training script
```

18.2 Master Component Traceability Table

Table 18.1 provides an exhaustive mapping of every primary Python module across the repository.

18.3 Developer's Diagnostic Guide: Where to Debug Specific Failures

When debugging anomalous behavior in multimodal satellite models:

1. **Global Color / Radiometric Drift:** If reconstructed clear fields appear discolored, inspect `CLEAR_CHANNEL_MEANS` in `models/ecrformer_adapted.py` (lines 62–66) and the optical scale factor in `data/cache_dataset.py`.

Component	Source File	Class / Function	Core Responsibility	Active in SOTA?
Dataset Loader	`data/cache_dataset.py`	`CachedSEN12MSCRDataset`	Loads `.npz` arrays; applies SAR dB normalization and 3-step mask softening.	Yes
Top-Level Adapter	`models/ecrformer_adapted.py`	`ECRformerAdaptedModel`	Splits 7-channel input; invokes U-Net; applies soft-mask output blend gate.	Yes
Decoupled Stem	`models/ecrformer_model.py`	`DecoupledEncoder`	Slices SAR ($2 \rightarrow 16$) and S2+M ($5 \rightarrow 16$) with reflection convs; concats to 32 ch.	Yes
Core U-Net	`models/ecrformer_model.py`	`ECRformerModel`	Orchestrates 4-stage encoder-decoder, skip links, and auxiliary projection heads.	Yes
Transposed Attn	`models/module.py`	`TransposedAttention`	Computes global channel-wise cross-covariance attention ($C \times C$).	Yes
Dilated Attn	`models/module.py`	`MultiDilateWindowAttention`	Computes multi-scale spatial attention	Yes

2. **Seam Artifacts around Cloud Edges:** If visible border lines appear, inspect lines 84–97 in `data/cache_dataset.py`. Verify that the Gaussian blur kernel size is 11 with $\sigma = 2.0$ and that morphological dilation is functioning.

3. **Blurry Over-Smoothed Outputs:** Inspect `losses/cloud_structure_loss.py`. Verify that the local standard deviation loss weight is active ($w = 0.15$) and that the high-pass residual factor in `losses/patchgan_loss.py` is set to 10.0.

4. **Adversarial Training Instability / Mode Collapse:** Inspect the discriminator in `losses/patchgan_loss.py`. Verify that every convolutional layer is wrapped with `spectral_norm` and that the generator adversarial weight does not exceed $w_{\text{adv}} = 0.015$.

18.4 Summary of Chapter 18

1. The codebase is organized into cleanly decoupled directories: `data/`, `models/`, `losses/`, and root training scripts.

2. Every component has an explicit, traceable responsibility verified against executable Python source files.

3. The master traceability table distinguishes between components active in the state-of-the-art sweetspot checkpoint versus experimental branches.

4. Targeted diagnostic pathways enable researchers to isolate color shifts, edge artifacts, blurriness, or training instability rapidly.

18.5 Practice Exercises

1. In which Python file is the soft-mask blending equation implemented?

2. Which module must you inspect if you want to modify the dilation rates used in multi-scale window attention?

3. Why is `losses/spectral_texture_loss.py` classified as an inactive experimental component in the Stage 4 pipeline?

Solutions

1. `models/ecrformer_adapted.py` (specifically line 110: `out = (1.0 - mask) * s2_cloudy + mask * pred_direct`).

2. `models/module.py` inside the `MultiDilateWindowAttention` class (lines 377–418).

3. While the 2D FFT spectral loss was implemented and tested during Stage 2, it was superseded in Stage 4 by the superior texture fidelity provided by the combination of local standard deviation matching, multi-scale signed gradients, and the 11-channel high-pass PatchGAN.

Chapter 19

Common Problems, Debugging, and Failure Analysis

19.1 The Reality of Satellite Deep Learning Debugging

In computer vision research, things rarely work on the first attempt. Training a multimodal, multi-scale transformer with adversarial loss involves dozens of moving parts. A single misaligned dimension, unnormalized channel, or inverted mask can corrupt months of computation.

This chapter presents a field-tested diagnostic guide to the 16 most common failure modes in multimodal satellite vision.

19.2 The 16 Critical Failure Modes and Proven Remedies

19.2.1 1. Optical Radiometric Normalization Mismatch

- **Symptom:** Outputs are entirely pure white, washed out, or completely pitch black.
- **Root Cause:** Sentinel-2 images are stored as 16-bit integers ($[0, 10000]$). Dividing by 255 (standard 8-bit web image normalization) causes all reflectance values to exceed 1.0 by a factor of 40!
- **Remedy:** Always normalize via $\text{clip}(\text{DN}/10000.0, 0.0, 1.0)$.

19.2.2 2. Swapped Channel Order

- **Symptom:** Model outputs surreal neon pinks or greens, and radar features are ignored.
- **Root Cause:** Python libraries handle channel orders differently (OpenCV uses BGR, PIL uses RGB, GeoTIFF uses arbitrary band indices). Slicing channels $0:2$ assuming they are SAR when they are actually Red/Green breaks the decoupled stem.
- **Remedy:** Enforce an explicit channel schema: `[VV, VH, B04, B03, B02, B08, Mask]`.

19.2.3 3. Memory Layout / Dimension Transposition

- **Symptom:** `RuntimeError: Expected 4D tensor with shape [B, C, H, W] but got [B, H, W, C]`.
- **Root Cause:** NumPy and TensorFlow use channel-last ($[B, H, W, C]$); PyTorch requires channel-first ($[B, C, H, W]$).
- **Remedy:** Apply `tensor.permute(0, 3, 1, 2).contiguous()`.

19.2.4 4. Inverted Cloud Mask

- **Symptom:** The final blended image contains clouds where clouds were, while clear areas are replaced by blurry neural predictions.
- **Root Cause:** Confusing 1 = cloud with 1 = clear. In our pipeline:

$$\hat{\mathbf{Y}} = (1.0 - \mathbf{M})\mathbf{X}_{S2} + \mathbf{M}\mathbf{P}_{\text{dir}} \tag{19.1}$$

  If $\mathbf{M} = 1$ on clear pixels, the formula discards the real clear pixels!
- **Remedy:** Always verify that $\mathbf{M} = 1$ corresponds to clouds and 0 to clear sky.

19.2.5 5. Misaligned SAR and Optical Imagery

- **Symptom:** Double coastlines, blurry roads, or ghosted city skylines.
- **Root Cause:** Sentinel-1 radar coordinates were not properly orthorectified with digital elevation models (DEM) or re-projected to match Sentinel-2's UTM grid.
- **Remedy:** Verify geocoding with sub-pixel cross-correlation before training.

19.2.6 6. Noisy or Seasonally Shifted Reference Targets

- **Symptom:** Training loss stops decreasing; model reconstructs green grass where winter snow should be.
- **Root Cause:** The clear-sky target was acquired in summer while the cloudy image was taken in winter. The model is penalized for failing to predict summer crop phenology from winter radar data!
- **Remedy:** Use scale-decoupled L_1 pooling $(4 \times 4, 8 \times 8)$ which reduces sensitivity to micro-temporal pixel phase variations.

19.2.7 7. Over-Smoothed Outputs (Low Detail Ratio)

- **Symptom:** Cloud-cleared regions have acceptable PSNR but look like smooth, blurry watercolor paintings (Detail Ratio < 0.50).
- **Root Cause:** Solely relying on pixel-wise L_1/L_2 losses (the conditional mean trap).
- **Remedy:** Activate local standard deviation matching (16×16), multi-scale signed gradients, and the 11-channel high-pass PatchGAN.

19.2.8 8. Global Color / Radiometric Drift

- **Symptom:** Entire reconstructed patches exhibit an unnatural yellow, purple, or dark cast.
- **Root Cause:** Final convolutional head bias initialized randomly with zero or Gaussian noise.
- **Remedy:** Initialize `final_conv.bias` to `CLEAR_CHANNEL_MEANS` ($[0.0847, 0.0919, 0.1127, 0.1884]$).

19.2.9 9. Unstable GAN Training and Mode Collapse

- **Symptom:** Discriminator loss drops to 0.0 within 50 steps; generator outputs catastrophic geometric checkerboard noise or repeating geometric artifacts.
- **Root Cause:** Discriminator is too strong, learning rates are unbalanced, or Lipschitz constraints are violated.
- **Remedy:** Apply Spectral Normalization to all discriminator layers, use mask-weighted Hinge loss, and reduce adversarial generator weight to $w_{\text{adv}} = 0.015$.

19.2.10 10. Loss Plateaus Early in Training

- **Symptom:** Loss decreases slightly in epoch 1 and remains completely flat for 20 epochs.
- **Root Cause:** Learning rate too low or model trapped in a poor local minimum; radar features not propagating.
- **Remedy:** Implement a $2\times$ differential learning rate on the SAR stem (2×10^{-4} vs. 1×10^{-4}) and verify LayerScale initialization ($\gamma = 0.01$).

19.2.11 11. Exploding or Vanishing Gradients

- **Symptom:** Loss outputs `NaN` (Not a Number) or weights freeze with zero updates.
- **Root Cause:** Extremely deep transformer attention layers without gradient clipping.
- **Remedy:** Apply `torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)` and Automatic Mixed Precision with `GradScaler`.

19.2.12 12. Overfitting (Train Loss Falls, Val Loss Rises)

- **Symptom:** Train loss reaches 0.01 while validation PSNR degrades from 30 dB $\rightarrow$ 25 dB.
- **Root Cause:** Model capacity is too high or data augmentation is absent.
- **Remedy:** Apply random horizontal/vertical flips, 90-degree rotations, synthetic opaque cloud core cutouts ($p = 0.65$), and weight decay (1×10^{-4}).

19.2.13 13. Spatial Data Leakage

- **Symptom:** Validation PSNR is artificially high (38 dB), but performance on a newly acquired external satellite tile is catastrophic (19 dB).
- **Root Cause:** Splitting dataset at the patch level rather than the geographical scene level.
- **Remedy:** Partition train, val, and test sets strictly by distinct ROI scene directories.

19.2.14 14. Metric Calculation Range Errors

- **Symptom:** PSNR reported as 8.2 dB or negative values despite visually flawless reconstructions.
- **Root Cause:** Evaluating a $[0, 1]$ normalized image with a PSNR function that assumes $\text{MAX}_I = 255.0$ (or vice-versa).
- **Remedy:** In $10 \log_{10}(\text{MAX}^2/\text{MSE})$, set $\text{MAX} = 1.0$ when inputs are in $[0, 1]$.

19.2.15 15. Visible Perimeter Seams in Blended Outputs

- **Symptom:** Sharp, unnatural cookie-cutter borders surround reconstructed clouds.
- **Root Cause:** Blending with an unblurred binary mask ($\{0, 1\}$).
- **Remedy:** Apply an 11×11 Gaussian blur ($\sigma = 2.0$) to the dilated cloud mask.

19.2.16 16. Out of Memory (OOM) GPU Crashes

- **Symptom:** `CUDA out of memory. Tried to allocate 2.40 GiB`.
- **Root Cause:** High batch size or spatial self-attention memory explosion ($\mathcal{O}((HW)^2)$).
- **Remedy:** Reduce batch size to 8 or 16, enable AMP (`torch.amp.autocast`), and ensure transposed channel attention (MDTA) is used instead of spatial attention.

19.3 Summary of Chapter 19

1. Normalization, channel ordering, and mask orientation must be verified with mathematical rigor before launching training.
2. Over-smoothing is eliminated by moving away from naive L_1/L_2 losses to gradient, texture std, and high-pass PatchGAN losses.
3. Mode collapse is prevented by Spectral Normalization and conservative adversarial weighting ($w_{\text{adv}} = 0.015$).
4. Generalization requires scene-level dataset splitting to avoid spatial memorization.

19.4 Practice Exercises

1. If a training loss abruptly outputs `NaN` in epoch 3, what are the first two diagnostic actions you should take?
2. How does calculating PSNR with MAX = 255.0 on a $[0, 1]$ image distort the resulting decibel score?
3. Why does a 3-step mask refinement (pool $\rightarrow$ dilate $\rightarrow$ Gaussian blur) resolve seam boundary artifacts?

Solutions

1. First, verify whether gradient norm clipping (`clip_grad_norm_`) was active, as unchecked gradient spikes produce infinities. Second, check whether any division by zero occurred (e.g., in cloud-masked losses without an $\epsilon = 10^{-8}$ denominator stabilizer) or if AMP `GradScaler` encountered severe overflow.
2. If MAX is incorrectly set to 255.0 instead of 1.0, the numerator inside the logarithm is $255^2 = 65{,}025$ times too large! The calculated PSNR will be artificially inflated by $+10 \log_{10}(65{,}025) \approx +\mathbf{48.13}$ dB!
3. Dilation guarantees that cloud shadows and penumbra haze are encompassed within the mask, while Gaussian softening creates an alpha-blend transition over several pixels, eliminating step-function discontinuities.

Chapter 20

Understanding the Novel Modifications

20.1 Beyond the Baseline: Why Architectural Adaptations Were Essential

The original ECRformer baseline was designed for general image restoration. When applied directly to multi-sensor satellite remote sensing with severe cloud contamination, standard training produced reconstructions with noticeable over-smoothing, retaining less than 50% of natural high-frequency detail.

To transform ECRformer into a robust, research-grade satellite reconstruction engine, six key innovations were engineered and integrated into the repository.

20.2 Innovation 1: The 11-Channel High-Pass Conditioned Discriminator

20.2.1 The Problem

Standard conditional GAN discriminators receive only the predicted optical image and condition images. In remote sensing, vast agricultural landscapes have low contrast, so a standard discriminator often focuses on overall color tone rather than micro-texture sharpness, allowing over-smoothed fields to pass as "real."

20.2.2 The Mathematical Solution

In `losses/patchgan_loss.py`, the discriminator is fed an explicit 10× magnified high-pass residual alongside base imagery:

$$\mathbf{R}_{\text{HP}} = 10.0 \times \left(\mathbf{Y} - \text{AvgPool}_{5\times5}(\mathbf{Y})\right) \tag{20.1}$$

The input tensor has 11 channels:

$$\mathbf{X}_{\text{disc}} = \left[3.0\mathbf{Y},\ 10.0(\mathbf{Y} - \mathcal{P}_5(\mathbf{Y})),\ \mathbf{X}_{S1},\ \mathbf{M}\right] \in \mathbb{R}^{B\times11\times H\times W}$$

20.2.3 Impact

The high-pass subtraction strips out macro-brightness, forcing the discriminator to examine high-frequency texture roughness directly. If the generator produces an over-smoothed smudge, its high-pass channels are flat, triggering immediate adversarial penalties that compel the generator to restore natural grain.

20.3 Innovation 2: Scale-Decoupled L_1 Supervision

20.3.1 The Problem

Due to orbital revisit schedules, the clear reference image is acquired on a different day than the cloudy image. High winds or seasonal shifts cause individual tree leaves or crop furrows to shift by a few pixels. Strict pixel-wise L_1 penalizes these natural micro-shifts, discouraging the model from generating sharp features.

20.3.2 The Mathematical Solution

We decouple L_1 error across three distinct spatial scales:

$$\mathcal{L}_{\text{scale}} = 0.35\|\hat{\mathbf{Y}} - \mathbf{Y}^*\|_1 + 0.40\|\mathcal{P}_4(\hat{\mathbf{Y}}) - \mathcal{P}_4(\mathbf{Y}^*)\|_1 + 0.25\|\mathcal{P}_8(\hat{\mathbf{Y}}) - \mathcal{P}_8(\mathbf{Y}^*)\|_1 \tag{20.2}$$

The 4×4 and 8×8 pooled terms enforce strict regional radiometric accuracy without penalizing sub-patch high-frequency phase shifts.

20.4 Innovation 3: Dual MDTA + MDWA Hybrid Attention in Every Block

20.4.1 The Problem

Standard vision transformers choose between spatial window attention (Swin, DilateFormer) or channel attention (Restormer). In satellite imagery, both are indispensable:

- Global cross-channel covariance is needed to correlate microwave polarizations with optical reflectances.
- Multi-scale spatial attention is needed to delineate geological boundaries.

20.4.2 The Architectural Solution

In `models/module.py` (lines 425–475), each `ECRformerBlock` contains **both** Transposed Attention (MDTA) and Multi-Dilated Window Attention (MDWA) executed in series, each with an independent LayerScale residual multiplier ($\gamma_1, \gamma_2 = 0.01$).

20.5 Innovation 4: Opaque Cloud Core Cutout Augmentation

20.5.1 The Problem

In semi-transparent clouds, optical sensors still receive faint optical photons. Models often learn to cheat by relying on residual optical signals rather than learning to decode Sentinel-1 radar features.

20.5.2 The Implementation

In `train_ecrformer_v3.py` and `v4`, we implemented an automated cutout augmentation:

- With probability $p = 0.65$, pixels where $\text{AvgPool}_{15\times15}(\mathbf{M}) > 0.8$ are identified as opaque cloud cores.
- The cloudy optical channels inside these cores are synthetically zeroed out or replaced with noise.
- The generator is forced to reconstruct the ground surface solely from Sentinel-1 radar data, dramatically strengthening radar-to-optical feature synthesis!

20.6 Innovation 5: $2\times$ Differential Learning Rate on SAR Stem

20.6.1 The Optimization Strategy

Because microwave backscatter contains speckle noise and different physical scattering mechanisms, standard optimizers tend to minimize loss by tuning optical filters while leaving radar weights sluggish. By assigning a $2\times$ higher learning rate (2×10^{-4} vs. 1×10^{-4}) to `stem.branch_0` and `encoder[0]`, the model rapidly converges to optimal microwave edge filters.

20.7 Innovation 6: Soft-Mask Blending Gate with Mean-Prior Initialization

20.7.1 The Blending Solution

Integrating the soft-mask equation:

$$\hat{\mathbf{Y}} = (1.0 - \mathbf{M}) \odot \mathbf{X}_{S2} + \mathbf{M} \odot \mathbf{P}_{\text{dir}} \tag{20.3}$$

directly inside the forward pass guarantees that clear-sky terrain is preserved with 100% original radiometric truth, while initializing the final convolutional bias to `CLEAR_CHANNEL_MEANS` ensures zero initial loss shock.

20.8 Summary of Chapter 20

1. The 11-channel high-pass discriminator isolates and enforces high-frequency texture realism.
2. Scale-decoupled L_1 loss accommodates micro-temporal phase shifts between multi-date acquisitions.
3. Dual MDTA + MDWA attention combines global channel covariance with multi-scale spatial receptive fields.
4. Opaque cloud core cutout forces the network to learn robust radar-to-optical translation.
5. A $2\times$ differential learning rate accelerates radar feature convergence.
6. Soft-mask blending with mean-prior initialization eliminates clear-sky distortion and stabilizes initial training.

20.9 Practice Exercises

1. Why does subtracting a 5×5 average pool from an image isolate its high-frequency textures?
2. What risk is introduced if the generator adversarial loss weight w_{adv} is set too high (e.g., $w_{\text{adv}} = 1.0$ instead of 0.015)?
3. How does opaque cloud core cutout prevent the neural network from "cheating" during training?

Solutions

1. An average pool is a low-pass spatial filter that averages neighboring pixels, smoothing away sharp transitions. Subtracting the smoothed low-pass image from the original leaves only the rapid spatial variations (edges, textures, noise), which constitute the high-frequency spectrum.

2. If w_{adv} is too large, the generator prioritizes fooling the discriminator over matching true ground reflectance, producing severe hallucinations, distorted land borders, and color artifacts. A conservative weight (0.015) allows the adversarial loss to add texture sharpness while reconstruction and perceptual losses maintain physical truth.

3. Under thin clouds, some optical light penetrates to the sensor. A model might learn to simply boost contrast on these faint optical traces rather than interpreting the radar data. Cutting out the optical core entirely forces the model to rely 100% on radar data to reconstruct the surface.

Chapter 21

Ablation Studies and Scientific Evaluation

21.1 The Scientific Method in Deep Learning Research

In academic machine learning, asserting that an architecture is "better" is meaningless without rigorous, controlled **ablation studies**. An ablation study systematically isolates each proposed component to prove whether performance gains stem from genuine architectural innovations or random statistical noise.

21.2 Systematic Experimental Evolution Across Four Stages

Table 21.1 summarizes the quantitative progression across the four major experimental iterations conducted in our repository, evaluated across held-out heavy-cloud validation scenes ($> 70\%$ cloud cover).

21.3 In-Depth Analysis of Ablation Results

21.3.1 Stage 1 vs. Stage 4: Deconstructing the "PSNR Trap"

Notice that Stage 1 achieved the highest raw PSNR (30.20 dB), yet suffered from catastrophic texture loss (Detail Ratio 0.3540). Because Stage 1 minimized pure L_1 error, the model learned to output smooth, safe conditional averages.

In Stage 4, adding the 11-channel high-pass PatchGAN and local standard deviation loss caused the Detail Ratio to surge from $\mathbf{0.3540} \rightarrow \mathbf{0.8576}$—a $+\mathbf{142}\%$ improvement in texture retention—with SSIM climbing from $0.8747 \rightarrow 0.8815$. The minor -0.23 dB shift in PSNR is a classic manifestation of the perception-distortion trade-off: slight sub-pixel phase adjustments yield vastly superior visual realism and physical fidelity!

21.3.2 The Impact of Intermediate SDFL Auxiliary Supervision

Ablating the auxiliary projection heads (`down_proj[0..2]` and `up_proj[0..2]`) showed that training took $2.4\times$ longer to reach equivalent validation loss. Without multi-scale supervision, early encoder layers experienced sluggish gradient updates during the first 10 epochs.

21.3.3 The Impact of Soft-Mask Blending

Evaluating predictions without the soft-mask blend gate—using the raw neural output $\mathbf{P}_{\text{dir}}$ globally—caused PSNR on clear-sky test patches to drop from 38.4 dB $\rightarrow$ 28.1 dB. Even subtle

Experimental Stage	Heavy PSNR (dB)	Heavy SSIM	Detail Ratio	Edge Corr	Reconstruction Character
Stage 1: Baseline ECRformer	**30.20**	0.8747	0.3540	0.2812	Severe over-smooth flat, painted ap-pear-ance; 65% tex-ture lost.
Stage 2: Spectral FFT Loss	29.85	0.8690	0.5120	0.3140	Modera tex-ture re-cov-ery; pe-ri-odic grid ar-ti-facts from Fourier ring-ing.
Stage 3: Structural/Perceptual	30.08	0.8792	0.6845	0.3725	Sharp bound-ary con-tours; strong veg-e-ta-tion con-trast; re-al-is-tic macro-scopic lay-out.
Stage 4: Sweetspot PatchGAN	29.97	**0.8815**	**0.8576**	**0.4153**	Near-total tex-ture preser-va-tion (> 85%); au-then-

neural imperfections degrade pristine optical observations. Soft-mask blending is mandatory to ensure clear pixels remain 100% untouched.

21.4 Computational Efficiency and Parameter Footprint

- **Active Generator Parameter Count:** 3,619,070 parameters (3.62 M).
- **Disk Checkpoint Size:** $\approx$ 34.3 MB (containing generator, EMA shadow weights, and PatchGAN discriminator).
- **Inference Speed on RTX 5070:** $<$ 12 ms per 128×128 patch ($>$ 80 patches/second).
- **Mobile / Edge Feasibility:** The standalone generator can be converted to LiteRT / ONNX ($\approx$ 14.5 MB in float32, or $<$ 4.0 MB in INT8 quantization), enabling real-time deployment on edge devices or Android ground stations!

21.5 Summary of Chapter 21

1. Controlled ablation benchmarks demonstrate how each architectural modification directly targets specific physical deficiencies.
2. Moving from Stage 1 to Stage 4 improved high-frequency detail retention from 35.4% to 85.8%.
3. The perception-distortion trade-off explains why human-preferred, photorealistic reconstructions can have marginally lower PSNR than artificially blurred predictions.
4. Auxiliary SDFL heads accelerate convergence by 2.4×.
5. The active 3.62M generator is computationally lightweight, fast, and primed for edge deployment.

21.6 Practice Exercises

1. Why is a higher PSNR not always proof that a cloud removal model is superior?
2. What quantitative metric in Table 21.1 best captures the transition from a blurry reconstruction to a sharp reconstruction?
3. Why does omitting the soft-mask blend gate significantly degrade clear-sky test scores?

Solutions

1. PSNR measures logarithmic pixel-wise squared error. An over-smoothed model that predicts the uniform conditional mean avoids extreme pixel penalties and can score a higher PSNR than a model that synthesizes sharp, authentic textures with slight sub-pixel phase offsets.
2. The **Detail Ratio** (Laplacian high-frequency energy retention) best captures texture sharpness, rising from 0.3540 in the blurry baseline to 0.8576 in the sharp GAN model.
3. The original clear optical pixels have zero error relative to the clear ground truth. Passing clear terrain through a generative neural network invariably introduces minor neural noise and color shifts, whereas soft-mask blending copies the true optical pixels with zero loss of fidelity.

Chapter 22

A Complete Worked Example

22.1 Synthesizing the Pipeline with Concrete Numbers

Throughout this textbook, we have examined remote sensing physics, tensor dimensions, attention mathematics, and loss formulations in isolation.

In this chapter, we assemble every concept into a single, complete **end-to-end numerical walkthrough**. We will take a tiny 2×2 pixel image, pass it through a miniature multimodal network, compute the composite loss, derive exact backpropagation gradients using the Chain Rule, and execute a parameter update.

22.2 The Micro-World Setup

To keep the manual arithmetic fully transparent while preserving multimodal properties, we define:

- Spatial grid: $H = 2, W = 2$ (2×2 pixels).
- Modalities:
  - Sentinel-1 SAR VV ($\mathbf{X}_{\text{SAR}} \in \mathbb{R}^{1\times2\times2}$).
  - Sentinel-2 Optical Red band ($\mathbf{X}_{\text{opt}} \in \mathbb{R}^{1\times2\times2}$).
  - Soft Cloud Mask ($\mathbf{M} \in \mathbb{R}^{1\times2\times2}$).
- Target ground truth: $\mathbf{Y}^* \in \mathbb{R}^{1\times2\times2}$.

22.2.1 Input Data Values

$$\mathbf{X}_{\text{SAR}} = \begin{bmatrix} 0.20 & 0.80 \\ 0.20 & 0.80 \end{bmatrix} \quad \text{(left col: calm water; right col: urban concrete)}$$

$$\mathbf{X}_{\text{opt}} = \begin{bmatrix} 0.10 & 0.95 \\ 0.10 & 0.95 \end{bmatrix} \quad \text{(left col: clear water; right col: obscured by cloud!)}$$

$$\mathbf{M} = \begin{bmatrix} 0.00 & 1.00 \\ 0.00 & 1.00 \end{bmatrix} \quad \text{(left col: clear sky; right col: thick cloud)}$$

$$\mathbf{Y}^* = \begin{bmatrix} 0.10 & 0.25 \\ 0.10 & 0.25 \end{bmatrix} \quad \text{(true ground truth: water on left, urban asphalt on right)}$$

22.3 The Miniature Multimodal Architecture

We construct a minimal model with two learnable parameters:

1. A radar stem weight w_{sar} and an optical stem weight w_{opt}:

$$\mathbf{P}_{\text{dir}} = w_{\text{sar}}\mathbf{X}_{\text{SAR}} + w_{\text{opt}}\mathbf{X}_{\text{opt}} \tag{22.1}$$

2. Initial parameter values: $w_{\text{sar}} = 0.50$, $w_{\text{opt}} = 0.50$.

3. Blended output:

$$\hat{\mathbf{Y}} = (1 - \mathbf{M}) \odot \mathbf{X}_{\text{opt}} + \mathbf{M} \odot \mathbf{P}_{\text{dir}} \tag{22.2}$$

4. Objective: L_1 reconstruction loss: $\mathcal{L} = \frac{1}{4}\sum_{i=1}^{4} |\hat{\mathbf{Y}}_i - \mathbf{Y}_i^*|$.

5. Learning rate: $\eta = 0.10$.

22.4 Step 1: The Forward Pass

First, compute the network's direct prediction $\mathbf{P}_{\text{dir}}$:

$$\begin{aligned}
\mathbf{P}_{\text{dir}}(0,0) &= (0.50)(0.20) + (0.50)(0.10) = 0.10 + 0.05 = \mathbf{0.15} \\
\mathbf{P}_{\text{dir}}(0,1) &= (0.50)(0.80) + (0.50)(0.95) = 0.40 + 0.475 = \mathbf{0.875} \\
\mathbf{P}_{\text{dir}}(1,0) &= (0.50)(0.20) + (0.50)(0.10) = \mathbf{0.15} \\
\mathbf{P}_{\text{dir}}(1,1) &= (0.50)(0.80) + (0.50)(0.95) = \mathbf{0.875}
\end{aligned}$$

$$\mathbf{P}_{\text{dir}} = \begin{bmatrix} 0.150 & 0.875 \\ 0.150 & 0.875 \end{bmatrix}$$

Next, apply the soft-mask blend gate $\hat{\mathbf{Y}} = (1 - \mathbf{M})\mathbf{X}_{\text{opt}} + \mathbf{M}\mathbf{P}_{\text{dir}}$:

$\hat{\mathbf{Y}}(0,0) = (1 - 0.0)(0.10) + (0.0)(0.15) = \mathbf{0.100}$ (exact clear ground truth preserved!)
$\hat{\mathbf{Y}}(0,1) = (1 - 1.0)(0.95) + (1.0)(0.875) = \mathbf{0.875}$ (cloudy pixel replaced by prediction)
$\hat{\mathbf{Y}}(1,0) = \mathbf{0.100}$
$\hat{\mathbf{Y}}(1,1) = \mathbf{0.875}$

$$\hat{\mathbf{Y}} = \begin{bmatrix} 0.100 & 0.875 \\ 0.100 & 0.875 \end{bmatrix}$$

22.5 Step 2: Loss Evaluation

Compare blended prediction $\hat{\mathbf{Y}}$ with ground-truth $\mathbf{Y}^*$:

$$\hat{\mathbf{Y}} - \mathbf{Y}^* = \begin{bmatrix} 0.100 - 0.100 & 0.875 - 0.250 \\ 0.100 - 0.100 & 0.875 - 0.250 \end{bmatrix} = \begin{bmatrix} 0.000 & +0.625 \\ 0.000 & +0.625 \end{bmatrix}$$

Evaluating the scalar L_1 loss:

$$\mathcal{L} = \frac{1}{4}\big(|0.000| + |0.625| + |0.000| + |0.625|\big) = \frac{1.250}{4} = \mathbf{0.3125}$$

22.6 Step 3: The Backward Pass (Autograd Derivatives)

We evaluate the gradient of $\mathcal{L}$ with respect to the blended output pixels:

$$\frac{\partial \mathcal{L}}{\partial \hat{\mathbf{Y}}} = \frac{1}{4}\text{sign}(\hat{\mathbf{Y}} - \mathbf{Y}^*) = \begin{bmatrix} 0.000 & +0.250 \\ 0.000 & +0.250 \end{bmatrix}$$

Now, propagate backward through the soft-mask gate:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{P}_{\text{dir}}} = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{Y}}} \odot \mathbf{M} = \begin{bmatrix} (0.000)(0.0) & (0.250)(1.0) \\ (0.000)(0.0) & (0.250)(1.0) \end{bmatrix} = \begin{bmatrix} 0.000 & +0.250 \\ 0.000 & +0.250 \end{bmatrix}$$

Notice that the left column has zero gradient because $\mathbf{M} = 0$!

Now, propagate backward into the weights:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w_{\text{sar}}} &= \sum_{i,j} \frac{\partial \mathcal{L}}{\partial \mathbf{P}_{\text{dir}}(i,j)} \cdot \mathbf{X}_{\text{SAR}}(i,j) \\ &= (0.000)(0.20) + (0.250)(0.80) + (0.000)(0.20) + (0.250)(0.80) \\ &= 0.000 + 0.200 + 0.000 + 0.200 = +\mathbf{0.400} \end{aligned}$$

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w_{\text{opt}}} &= \sum_{i,j} \frac{\partial \mathcal{L}}{\partial \mathbf{P}_{\text{dir}}(i,j)} \cdot \mathbf{X}_{\text{opt}}(i,j) \\ &= (0.000)(0.10) + (0.250)(0.95) + (0.000)(0.10) + (0.250)(0.95) \\ &= 0.000 + 0.2375 + 0.000 + 0.2375 = +\mathbf{0.475} \end{aligned}$$

22.7 Step 4: The Optimizer Parameter Update

Both gradients are positive, meaning the predictions were too bright and the weights must decrease:

$$w_{\text{sar, new}} = w_{\text{sar}} - \eta \frac{\partial \mathcal{L}}{\partial w_{\text{sar}}} = 0.500 - (0.10)(+0.400) = 0.500 - 0.040 = \mathbf{0.460}$$

$$w_{\text{opt, new}} = w_{\text{opt}} - \eta \frac{\partial \mathcal{L}}{\partial w_{\text{opt}}} = 0.500 - (0.10)(+0.475) = 0.500 - 0.0475 = \mathbf{0.4525}$$

22.8 Step 5: Verifying the Update on Step 2

Let us test the updated weights on a new forward pass:

$$\mathbf{P}_{\text{dir, new}}(0,1) = (0.460)(0.80) + (0.4525)(0.95) = 0.368 + 0.4299 = \mathbf{0.7979}$$

The error on the cloudy pixel dropped from $|0.875 - 0.25| = 0.625$ down to $|0.7979 - 0.25| = \mathbf{0.5479}$! The new loss is:

$$\mathcal{L}_{\text{new}} = \frac{1}{4}(0 + 0.5479 + 0 + 0.5479) = \mathbf{0.2739} < 0.3125$$

The system learned!

22.9 Summary of Chapter 22

1. Concrete matrix arithmetic traces the exact pathway from raw numbers through the blend gate to backpropagation.
2. The mask multiplication $\frac{\partial \mathcal{L}}{\partial \mathbf{P}_{\text{dir}}} = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{Y}}} \odot \mathbf{M}$ zeroes out upstream gradients on clear pixels.
3. Gradients accumulate the sensitivity of every cloudy pixel to each internal weight.
4. Gradient descent strictly reduces reconstruction error on the subsequent step.

22.10 Practice Exercises

1. In Step 3, why was the gradient with respect to the clear pixel $(0, 0)$ equal to zero?
2. If we had used Mean Squared Error ($\mathcal{L} = \frac{1}{2}(\hat{y} - y^*)^2$) instead of L_1, what would have been the derivative $\frac{\partial \mathcal{L}}{\partial \hat{y}_{(0,1)}}$?
3. If the radar weight received a $2\times$ differential learning rate ($\eta_{\text{sar}} = 0.20$), calculate the updated $w_{\text{sar, new}}$.

Solutions

1. At pixel $(0, 0)$, the mask value was $\mathbf{M} = 0.0$. Multiplying by $\mathbf{M}$ zeroes the gradient flowing to $\mathbf{P}_{\text{dir}}$. Because the original optical pixel already matched the target ($\hat{y} = y^* = 0.10$), the reconstruction error was also zero.
2. For MSE: $\frac{\partial \mathcal{L}}{\partial \hat{y}} = (\hat{y} - y^*) = (0.875 - 0.250) = +\mathbf{0.625}$.
3. $w_{\text{sar, new}} = 0.500 - (0.20)(+0.400) = 0.500 - 0.080 = \mathbf{0.420}$.

Chapter 23

Frequently Asked Questions

23.1 Core Conceptual and Practical Questions

Q1: Why can a neural network not simply copy the cloud-free target?

Because during real-world inference and production deployment, **the cloud-free target does not exist!** If we already had the cloud-free optical image, we would have no need for cloud removal! The cloud-free target is only provided during training to calculate loss gradients. The network must learn generalizable physical mappings from radar and context so it can reconstruct terrain when no optical target is available.

Q2: Why does a model need Sentinel-1 SAR data?

Optical light cannot penetrate thick clouds. An optical-only model is performing blind inpainting: it has zero evidence of whether a mountain, a lake, or a city lies beneath an opaque cloud. Sentinel-1 microwave radar penetrates clouds effortlessly, providing real-time measurements of surface roughness, moisture, and geometry that remove physical ambiguity.

Q3: Why do we need an encoder and a decoder?

The encoder progressively downsamples feature maps to compress macroscopic semantic context (e.g., recognizing that an entire valley is agricultural). The decoder upsamples this semantic blueprint back to full spatial resolution, while skip connections reintroduce high-resolution edges that were lost during downsampling.

Q4: Why use attention if CNNs already detect features?

Convolutional kernels only inspect local neighborhoods (3×3 pixels). To correlate an agricultural field on the north side of a wide cloud with an identical field on the south side, a CNN requires dozens of layers to build receptive field overlap. Attention mechanisms allow every pixel or channel to communicate directly across the entire image in a single computational step.

Q5: What does a transformer learn in satellite vision?

Transformers learn dynamic correlation patterns. In channel-wise attention (MDTA), the model learns how radar polarization channels correlate with optical reflectance bands across different biomes. In spatial window attention (MDWA), it learns multi-scale spatial texture continuity across cloud boundaries.

Q6: Why does a loss function matter so much?

The loss function defines the objective of optimization. If you use Mean Squared Error (L_2), the mathematical minimum is the conditional average of all possible ground surfaces, which guarantees an over-smoothed, blurry output. To obtain sharp textures, the loss function must penalize blurred gradients, local variance loss, and unnatural frequencies.

Q7: Why do gradients flow backward rather than forward?

Because the error (loss) is calculated at the very end of the network (the output). By applying the multivariable calculus Chain Rule, we must work backward from the output to calculate how changes in early weights affect the final error.

Q8: Why are there multiple loss functions combined in training?

No single mathematical loss captures all aspects of satellite imagery. Scale-decoupled L_1 ensures radiometric accuracy; multi-scale signed gradients ensure sharp boundaries; local standard deviation matching ensures surface roughness; VGG perceptual loss ensures semantic realism; and adversarial PatchGAN loss forces the synthesis of natural high-frequency textures.

Q9: Why does training require tens of thousands of examples?

A neural network with 3.6 million parameters has immense flexibility. If trained on only 50 images, it would simply memorize those specific fields (overfitting). Training on 33,468 patches spanning diverse seasons, climates, and continents forces the network to learn invariant physical relationships between microwave backscatter and optical reflectance.

Q10: What is the fundamental difference between training and inference?

During training, the model processes cloudy inputs, compares predictions against clear-sky targets, computes loss gradients, and updates its weights via backpropagation. During inference, clear targets are absent, autograd is disabled (`torch.no_grad()`), dropout is turned off (`model.eval()`), EMA weights are loaded, and the model outputs finalized reconstructed imagery.

Q11: Why can a model produce blurry watercolor-like results?

This is the "conditional mean trap." When a model is trained primarily with L_1 or L_2 pixel losses under high uncertainty, predicting a blurry average incurs less numerical penalty than predicting sharp textures with slight positional phase errors. Activating local variance matching and high-pass PatchGAN discriminators eliminates this flaw.

Q12: Why does a cloud mask have to be spatially aligned with the image?

If a cloud mask is shifted by even two pixels, the soft-mask blending gate $\hat{\mathbf{Y}} = (1-\mathbf{M})\mathbf{X}_{S2}+\mathbf{M}\mathbf{P}_{\text{dir}}$ will copy thick cloudy pixels into the final image while cutting out real ground pixels. Spatial alignment must be exact.

Q13: Why is the reconstructed output not guaranteed to be the exact original ground?

When an opaque cloud completely obscures an area, microscopic surface details (such as the exact arrangement of cars in a parking lot) do not produce distinct microwave signatures at 10-meter SAR resolution. The network reconstructs a physically consistent, highly plausible

surface matching the radar and contextual evidence, but cannot retrieve microscopic non-radar optical phenomena.

Q14: What is the difference between a GAN and a Transformer?

A Transformer is a **network architecture** utilizing attention mechanisms to model dependencies. A GAN is a **training framework** involving two competing networks (Generator and Discriminator). In our project, ECRformer is the Transformer that acts as the Generator inside a conditional GAN framework!

Q15: Can a GAN replace ECRformer?

No. The GAN is the game-theoretic training protocol, not a specific backbone. ECRformer-Light serves as the generator network inside the GAN. Removing ECRformer would require replacing it with an alternative generator backbone (such as a pure CNN or ResNet).

Q16: Does the model learn from the cloud mask directly?

Yes. The cloud mask enters the network as channel 6 in the 7-channel input tensor. This informs the encoder which spatial regions are obscured by clouds, allowing attention layers to discount cloudy optical pixels and prioritize radar signals.

Q17: Does the model use the mask during inference?

Yes. The mask is used both as an input conditioning channel and as the blending gate that seamlessly preserves original clear optical pixels while inserting reconstructed terrain under clouds.

Q18: Which modules contribute to the final prediction?

The decoupled stem, all 3 encoder stages, all 3 downsamplers, the Restormer bottleneck, all 3 upsamplers, all 3 decoder stages, the final convolutional head, and the soft-mask blend gate all contribute directly. Auxiliary projection heads (`down_proj`, `up_proj`) contribute to gradient flow during training but are discarded during inference.

Q19: Which modifications have actually been tested in our repository?

We have empirically tested: (1) Stage 1 Baseline ECRformer with SDFL; (2) Stage 2 Shift-Invariant 2D FFT Loss; (3) Stage 3 Structural Perceptual Loss with Opaque Cutout; and (4) Stage 4 11-Channel High-Pass Conditioned PatchGAN with Scale-Decoupled L_1. Stage 4 produced our premier sweetspot model (`ecrformer_sweetspot_best`).

23.2 Summary of Chapter 23

1. Multimodal satellite reconstruction relies on the physical synergy between cloud-penetrating radar and high-resolution optical bands.
2. Blurring is an artifact of naive objective functions, overcome through high-pass adversarial and variance losses.
3. Transformers provide the architectural backbone, while GANs provide the competitive training framework.
4. Mask conditioning and soft-mask blending are separate operations with distinct physical roles.

23.3 Practice Exercises

1. What is the difference between an architecture (like ECRformer) and an optimization framework (like a GAN)?

2. Why does an optical-only model fail under thick opaque clouds regardless of how deep its neural network is?

3. Which components of `ECRformerAdaptedModel` are active during inference versus only active during training?

Solutions

1. An architecture specifies the internal layers, tensor shapes, and mathematical operations used to transform inputs into outputs (e.g., convolutions, attention blocks, linear layers). An optimization framework defines how networks are trained and supervised (e.g., supervised loss minimization vs. adversarial minimax competition between a generator and discriminator).

2. Under completely opaque clouds, optical photons reflected from the ground are 100% scattered or absorbed before reaching the satellite sensor. The data received by the optical sensor contains zero ground information; therefore, no mathematical operation on optical data alone can recover the ground surface without guessing.

3. Active during inference: the Decoupled Stem, Encoders 0–2, Downsamplers 0–2, Bottleneck, Upsamplers 0–2, Decoders 0–2, Final Conv Head, and Soft-Mask Blend Gate. Active only during training: the Discriminator network and the auxiliary projection heads (`down_proj[0..2]` and `up_proj[0..2]`).

Chapter 24

Glossary and Learning Roadmap

24.1 Comprehensive Technical Glossary

This glossary provides concise, unambiguous definitions of every major concept and term encountered across this textbook:

Activation Function: A non-linear mathematical operation (such as ReLU, GELU, or LeakyReLU) applied element-wise to pre-activations, enabling neural networks to learn non-linear physical relationships.

Adversarial Loss: A loss formulation derived from game theory where a generator is penalized based on a discriminator's ability to distinguish synthetic reconstructions from real sensor acquisitions.

Attention: A computational mechanism that dynamically calculates similarity scores between queries (Q) and keys (K) to aggregate contextually relevant values (V) across an entire image or feature channel.

Backpropagation: The algorithm that applies the multivariable calculus Chain Rule in reverse topological order through a computational graph to compute partial derivatives of a scalar loss with respect to all trainable network weights.

Batch: A collection of independent data samples (patches) grouped together and processed concurrently in GPU memory during a single forward and backward optimization step.

Bias: An adjustable additive scalar or vector parameter in a neural layer that shifts the activation threshold independently of the input values.

Cloud Mask: A spatial map indicating the location, thickness, or probability of clouds and atmospheric haze over an image grid, where 0.0 = clear and 1.0 = cloudy.

Convolution: A mathematical operation that slides a small learnable matrix of weights (kernel/filter) across an input grid, extracting localized spatial features with translation equivariance.

Decoder: The expansive sub-network in a U-Net architecture that progressively upsamples compressed low-resolution semantic representations back to full image resolution.

Discriminator: The detective network in a GAN framework trained to distinguish real satellite acquisitions from synthetic generator reconstructions.

Encoder: The contracting sub-network in a U-Net architecture that progressively downsamples spatial resolution to extract abstract, macroscopic semantic context.

Epoch: One complete iteration during training where every training sample in the dataset has been processed by the model exactly once.

Feature Map: A 2D or 3D tensor of intermediate neural activations representing specific extracted visual or physical patterns (such as edges, textures, or crop types).

Forward Propagation: The execution pass that feeds input tensors through consecutive layers of matrix multiplications and activations to generate a prediction.

Generator: The neural network in a GAN framework that produces synthetic images (e.g., ECRformer-Light generating clear optical images from SAR and cloudy inputs).

Gradient: The vector of partial derivatives ($\nabla_\theta \mathcal{L}$) pointing in the direction of steepest increase of the loss function.

Loss Function: A scalar mathematical objective that measures the discrepancy between a model's prediction and the desired ground-truth target.

Normalization: The process of rescaling numerical values into a standard bounded dynamic range (e.g., $[0, 1]$ or $[-1, 1]$) to stabilize activations and gradient flow.

Optimizer: The mathematical algorithm (such as AdamW or SGD) that adjusts trainable parameters using computed gradients to minimize the loss function.

PatchGAN: A fully convolutional discriminator that evaluates image realism across local $N \times N$ spatial patches rather than outputting a single global scalar.

PSNR (Peak Signal-to-Noise Ratio): A logarithmic metric measuring the ratio between the maximum valid physical signal power and mean squared reconstruction error, expressed in decibels (dB).

SAR (Synthetic Aperture Radar): An active microwave remote sensing technology that transmits radar pulses and measures backscattered echoes, penetrating clouds, rain, and nighttime darkness.

Sentinel-1: The European Space Agency's C-band radar satellite constellation capturing dual-polarization (VV, VH) backscatter.

Sentinel-2: The European Space Agency's optical multispectral satellite constellation capturing 13 bands from visible blue to shortwave infrared.

SSIM (Structural Similarity Index Measure): A perceptual quality metric evaluating structural pattern consistency, luminance, and contrast between two images.

Tensor: A multi-dimensional numerical array in deep learning frameworks (e.g., a 4D tensor with shape $[B, C, H, W]$).

Transformer: A neural network architecture that relies fundamentally on self-attention mechanisms rather than recurrence to model dependencies across sequences or images.

Weight: A trainable multiplicative parameter within a neural network that scales the strength of incoming signals.

24.2 Five-Stage Learning Roadmap for Aspiring Researchers

To advance from a beginner to an autonomous remote sensing deep learning researcher, follow this curated curriculum:

1. **Phase 1: PyTorch Foundations and Tensor Calculus**
    - Master multidimensional slicing, broadcasting, and GPU tensor allocation in PyTorch.
    - Build and train a small multi-layer perceptron from scratch using manual autograd.
2. **Phase 2: Remote Sensing Physics and Geospatial Processing**
    - Study electromagnetic scattering principles (Rayleigh vs. Mie vs. microwave scattering).
    - Understand Sentinel-1 SAR polarimetry (VV surface roughness vs. VH volume canopy scattering) and Sentinel-2 multispectral indices (NDVI, NDWI).
3. **Phase 3: Convolutional U-Nets and Modern Vision Transformers**
    - Implement classic U-Net with skip connections, PixelShuffle upsampling, and reflection padding.
    - Study transposed channel attention (Restormer MDTA) and multi-dilated window attention (DilateFormer).
4. **Phase 4: Objective Formulation and Adversarial Game Theory**
    - Understand why L_1 and L_2 losses fail on ambiguous inverse problems.
    - Implement PatchGAN discriminators with Spectral Normalization, high-pass residual conditioning, and hinge losses.
5. **Phase 5: Edge Optimization and Research Defense**
    - Quantize and export trained transformer checkpoints to LiteRT and ONNX for mobile and embedded platforms.
    - Design rigorous ablation studies and defend architectural choices against peer review.

24.3 Summary of Chapter 24

1. The technical glossary provides unified definitions bridging computer science, remote sensing physics, and deep learning.
2. The 5-stage learning roadmap establishes a structured educational progression from basic tensors to publishable research defense.

24.4 Practice Exercises

1. Contrast the physical information captured by Sentinel-1 SAR versus Sentinel-2 optical imagery.
2. Explain in your own words the difference between an `epoch` and a `batch`.
3. Why is Spectral Normalization essential when training generative adversarial networks?

Solutions

1. Sentinel-2 optical imagery measures passive solar reflectance across visible and near-infrared bands, capturing surface color, chemical pigments, and vegetation photosynthesis, but is blocked by clouds. Sentinel-1 SAR transmits active C-band microwaves that penetrate clouds to measure physical surface roughness, geometric structures, moisture, and dielectric properties.

2. A **batch** is a small subset of samples (e.g., 16 patches) processed together in a single forward-backward pass. An **epoch** is completed only when the optimizer has processed every single batch in the entire training dataset (e.g., all 33,468 patches) once.

3. Spectral normalization enforces 1-Lipschitz continuity on the discriminator by dividing weight matrices by their spectral norm. This prevents discriminator gradients from exploding and guarantees bounded, stable gradient feedback to the generator, eliminating mode collapse.

Implementation Details and Traceability Audit

24.5 Component Implementation Status Matrix

To maintain complete scientific and engineering integrity, Table 24.1 explicitly categorizes every architectural and experimental module according to its verified execution status in the codebase.

24.6 Exact Parameter Breakdown Audit

Table 24.2 provides the exact parameter counts verified directly from the saved sweetspot checkpoint dictionary (`ecrformer_sweetspot_best/best_model.pth`).

24.7 Detailed Parameter Counts of Generator Sub-Modules

- **Decoupled Input Stem:** 5,520 parameters (0.15%).
- **Encoder Stage 0 ($C = 32$):** 33,962 parameters (0.94%).
- **Downsampler 0 ($32 \to 64$):** 10,798 parameters (0.30%).
- **Encoder Stage 1 ($C = 64$):** 125,420 parameters (3.47%).
- **Downsampler 1 ($64 \to 128$):** 32,494 parameters (0.90%).
- **Encoder Stage 2 ($C = 128$):** 241,641 parameters (6.68%).
- **Downsampler 2 ($128 \to 256$):** 64,522 parameters (1.78%).
- **Restormer Bottleneck ($C = 256$):** 672,518 parameters (18.58%).
- **Upsampler 0 ($256 \to 128$):** 1,189,390 parameters (32.86%).
- **Decoder Stage 0 ($C = 128$):** 536,681 parameters (14.83%).
- **Upsampler 1 ($128 \to 64$):** 307,854 parameters (8.51%).
- **Decoder Stage 1 ($C = 64$):** 199,212 parameters (5.50%).
- **Upsampler 2 ($64 \to 32$):** 54,026 parameters (1.49%).
- **Decoder Stage 2 ($C = 64$):** 125,420 parameters (3.47%).
- **Final Reconstruction Head:** 2,308 parameters (0.06%).
- **Auxiliary Heads (`down_proj`, `up_proj`):** 17,304 parameters (0.48%).
- **Total Generator Trainable Parameters: 3,619,070 parameters**.

Category	Component Name	Source File	Verified Implementation Status
Confirmed Active	`CachedSEN12MSCRDataset`	`data/cache_dataset.py`	Active in all runs; loads `.npz` arrays with on-the-fly SAR dB norm and 3-step mask softening.
Confirmed Active	`ECRformerAdaptedModel`	`models/ecrformer_adapted.py`	Active; 7-channel input slicing, 4-stage U-Net forward pass, and soft-mask output blend gate.
Confirmed Active	`DecoupledEncoder`	`models/ecrformer_model.py`	Active; independent reflection convs on SAR ($2 \rightarrow 16$)

Sub-Network	**State Dictionary Key**	**Tensors**	**Parameters**	**Role in System**
Active Generator	`model_state_dict`	277	**3,619,070**	Active neural model executing forward inference on the GPU.
EMA Shadow Weights	`ema_state_dict`	277	**3,619,070**	Temporal moving average ($\beta = 0.999$) used for validation and inference.
PatchGAN Discriminator	`disc_state_dict`	17	**1,260,737**	11-channel discriminator used during adversarial training.
Spectral Norm Buffers	`disc` (`_u, _v`)	10	8,561	Persistent singular vector buffers maintaining 1-Lipschitz continuity.
Total Saved Checkpoint	`best_model.pth`	581	**8,507,438**	Complete state dictionary archived on disk ($\approx$ 34.3 MB).

Table 24.2: Verified parameter audit of `best_model.pth`. The active generator contains 3.62 million parameters; previous reports of $\approx$ 8.3M parameters resulted from summing the generator, EMA copy, and discriminator together.

Bibliography

[1] Patrick Ebel, Andrea Meraner, Michael Schmitt, and Xiao Xiang Zhu. Multisensor data fusion for cloud removal in global and all-season Sentinel-2 imagery. *IEEE Transactions on Geoscience and Remote Sensing*, 59(7):5866–5878, 2021.

[2] Alaaeldin El-Nouby, Hugo Touvron, Mathilde Caron, Piotr Bojanowski, Matthijs Douze, Armand Joulin, Ivan Laptev, Natalia Neverova, Gabriel Synnaeve, Jakob Verbeek, and Hervé Jégou. XCiT: Cross-covariance image transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 20014–20027, 2021.

[3] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A. Efros. Image-to-image translation with conditional adversarial networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1125–1134, 2017.

[4] Jieru Jiao, Yu-Ming Tang, Kun-Yu Lin, Yipeng Gao, AJ Ma, Yaowei Wang, and Wei-Shi Zheng. DilateFormer: Multi-scale dilated transformer for visual recognition. *IEEE Transactions on Multimedia*, 25:8906–8919, 2023.

[5] Justin Johnson, Alexandre Alahi, and Li Fei-Fei. Perceptual losses for real-time style transfer and super-resolution. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 694–711, 2016.

[6] Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks. In *International Conference on Learning Representations (ICLR)*, 2018.

[7] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241, 2015.

[8] Wenzhe Shi, Jose Caballero, Ferenc Huszár, Johannes Totz, Andrew P. Aitken, Rob Bishop, Daniel Rueckert, and Zehan Wang. Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1874–1883, 2016.

[9] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations (ICLR)*, 2015.

[10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[11] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.

[12] Syed Waqas Zamir, Aditya Arora, Salman Khan, Munawar Hayat, Fahad Shahbaz Khan, and Ming-Hsuan Yang. Restormer: Efficient transformer for high-resolution image restoration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5728–5739, 2022.